

Statistik 2: Inferenz und Modellierung

Sang-Min Park

2026-07-10

Inhaltsverzeichnis

Vorwort	5
Entstehungsgeschichte dieses Skripts	5
Über den Einsatz von KI in diesem Kurs: Von taktischer zu strategischer Programmierung	6
1 Grundlagen der Inferenz mit Stichproben	7
2 Grundlagen der Inferenz mit Stichproben	8
2.1 Die Stichprobenverteilung und der Zentrale Grenzwertsatz	8
2.2 Arbeiten mit der Normalverteilung in R	10
2.2.1 Wahrscheinlichkeiten und Quantile ermitteln	10
2.3 Von der Variabilität zum Konfidenzintervall	11
2.3.1 Fallstudie: Stents und Schlaganfallrisiko	12
2.4 Zusammenfassung	12
3 Angewandte Inferenz	14
4 Angewandte Inferenz	15
4.1 Das universelle Prinzip: Die Anatomie der Teststatistik	15
4.2 Inferenz für Anteile (Proportionen)	16
4.2.1 Voraussetzungen für Proportionen	16
4.2.2 Operative Umsetzung in R (<code>prop.test</code>)	16
4.3 Inferenz für Mittelwerte	18
4.3.1 Bedingungen für Mittelwerte (Das “LUNG”-Konzept)	18
4.3.2 Operative Umsetzung in R (<code>t.test</code>)	19
4.4 Zusammenfassung der Entscheidungsstrukturen	23
5 Lineare Regression (OLS)	24
6 Lineare Regression (OLS)	25
6.1 Die einfache lineare Regression	25
6.1.1 Visuelle Intuition und das Modell	25
6.1.2 Operative Umsetzung in R (<code>lm</code>)	26
6.2 Das Konzept der Marginalen Effekte	27
6.3 Multiple Lineare Regression und Interaktionen	28
6.3.1 Fallstudie: Zinssätze für Kredite	28
6.3.2 Interaktionseffekte (Wenn Ceteris Paribus bricht)	28

6.4	Modellgüte und LUNG-Bedingungen	31
6.4.1	Das Bestimmtheitsmaß (R^2)	32
6.4.2	Die LUNG-Bedingungen prüfen	32
7	Logistische Regression	34
8	Logistische Regression	35
8.1	Warum die lineare Regression hier scheitert	35
8.2	Das Logistische Modell	36
8.2.1	Operative Umsetzung in R (<code>glm</code>)	37
8.3	Marginale Effekte in der logistischen Regression	38
8.3.1	Wahrscheinlichkeiten berechnen	39
8.3.2	Fallstudie: Die Auswirkung von Diskriminierung	39
8.4	Zusammenfassung der Entscheidungsstrukturen	40
9	Modellselektion	41
10	Modellselektion	42
10.1	Das Problem: Overfitting (Überanpassung)	42
10.2	Kennzahlen für die Modellauswahl	44
10.2.1	1. Das Angepasste R^2 (Für Lineare Regression)	44
10.2.2	2. Das AIC (Für Logistische Regression)	44
10.3	Die ultimative Prüfung: Kreuzvalidierung (Cross-Validation)	45
10.3.1	Workflow: k -fache Kreuzvalidierung	45
10.3.2	Praxis-Beispiel: Pinguin-Körpermasse	45
10.4	Fazit: Die Strategie eines Data Scientists	48
11	Fragebogendaten und Latente Konstrukte	49
12	Fragebogendaten und Latente Konstrukte	50
12.1	Der Paradigmenwechsel: Unsupervised Learning	50
12.1.1	Das Praxis-Problem: Multikollinearität im Fragebogen	51
12.1.2	Die Lösung: Latente Konstrukte	51
12.2	Durchführung einer Faktoranalyse in R	51
12.2.1	Schritt 1: Wie viele Faktoren gibt es? (Der Scree-Plot)	51
12.2.2	Schritt 2: Die Faktoren extrahieren (Faktorladungen)	53
12.2.3	Schritt 3: Faktor-Scores für die Regression berechnen	54
12.3	Zusammenfassung: Die Daten-Pipeline	55
13	Anhang: Quarto Survival Guide	56
Anhang:	Quarto Survival Guide	57
13.1	Das Skelett (Der YAML-Header)	57
13.2	Die Markdown-Basics (Textformatierung)	58

13.3 Chunk-Kontrolle (Die Magie)	59
13.4 Best Practices für Abgaben	59

Vorwort

Willkommen zum Skript für das Modul **Statistik 2: Inferenz und Modellierung**.

Dieses Skript begleitet Sie durch den Kurs und verbindet die statistische Theorie direkt mit der praktischen Anwendung in R.

Die Schwerpunkte dieses Skripts liegen auf:

- Dem Verständnis von Stichprobenverteilungen und Konfidenzintervallen.
- Der sicheren Auswahl und Anwendung von Hypothesentests (Anteile und Mittelwerte).
- Der Modellierung komplexer Zusammenhänge mittels linearer und logistischer Regression.

Nutzen Sie das Inhaltsverzeichnis auf der linken Seite, um durch die einzelnen Kapitel zu navigieren.

Nutzen Sie die Links auf der rechten Seite, um innerhalb eines Kapitels zu navigieren.

Der öffentliche Link zu diesem Skript ist: <https://019f2d55-5e33-9022-b065-c4581443be0d.share.connect.posit.cloud/>

Eine PDF Version dieses Skripts verteile ich auch über den Moodlekurs.

Entstehungsgeschichte dieses Skripts

- Inhaltlich baut dieses Skript auf dem Open Source Lehrbuch “Introduction to Modern Statistics” von Mine Çetinkaya-Rundel and Johanna Hardin auf.
- Sie finden es online unter <https://openintro-ims.netlify.app/>
- Ich habe das Lehrbuch auszugsweise mit KI-Unterstützung ins Deutsche übersetzt und in mehreren Überarbeitungsschleifen angepasst.
- Wenn Sie neben dem eher technischen Zugang dieses Kurses auch nicht-technische Darstellungen lesen möchten, empfehle ich zwei Bücher:
 - “Naked Statistics” von Charles Wheelan,
 - “The Art of Statistics” von David Spiegelhalter.

Über den Einsatz von KI in diesem Kurs: Von taktischer zu strategischer Programmierung

Wenn Sie Statistik und Datenanalyse lernen, stehen Sie vor zwei völlig unterschiedlichen Herausforderungen:

1. Taktische Programmierung (Die Handwerksarbeit): Wie lautet der exakte R-Befehl? Wo fehlt ein Komma? Warum wirft R die Fehlermeldung “object not found”?
2. Strategische Programmierung (Die Denk-Arbeit): Welches statistische Modell beantwortet unsere Forschungsfrage? Sind die Annahmen des Modells erfüllt? Wie übersetzen wir den p-Wert in eine Handlungsempfehlung für das Management?

In der Vergangenheit verbrachten Studierende (und Analysten) 80 % ihrer Zeit mit taktischen Problemen und Frust-Google. Das ist größtenteils vorbei.

In diesem Kurs ist der Einsatz von KI (wie ChatGPT oder GitHub Copilot) als taktischer Assistent ausdrücklich erlaubt und erwünscht. Die KI ist Ihre Flotte von Junior-Entwicklern, die den Code für Sie tippt und Fehler sucht. Ihr Job auf dem Arbeitsmarkt der Zukunft ist es nicht, ein menschliches Syntax-Lexikon zu sein. Ihr Job ist das strategische Review: Sie geben der KI die Architektur vor, Sie kontrollieren die Annahmen, und Sie verantworten die Interpretation. Wer blind kopiert, wird scheitern. Wer die KI strategisch steuert, wird exzellente Analysen produzieren.

1 Grundlagen der Inferenz mit Stichproben

2 Grundlagen der Inferenz mit Stichproben

i Kompetenzziele für dieses Kapitel

Nach der Bearbeitung dieses Kapitels können Sie:

1. **Konzept:** Erklären, warum Stichprobenstatistiken (wie der Mittelwert $\bar{x}$ oder Anteil $\hat{p}$) variieren und was eine *Stichprobenverteilung* ist.
2. **Theorie:** Den Zentralen Grenzwertsatz (ZGS) anwenden, um die Variabilität von Stichprobendaten theoretisch zu modellieren.
3. **Anwendung:** R-Funktionen (`pnorm`, `qnorm`) nutzen, um Wahrscheinlichkeiten und Perzentile unter der Normalverteilung zu berechnen, anstatt Formeln manuell aufzulösen.
4. **Synthese:** Ein 95%-Konfidenzintervall konstruieren und dessen Bedeutung für eine Grundgesamtheit statistisch korrekt interpretieren.

Zu den wichtigsten Konzepten der Statistik gehört es, Schlussfolgerungen über eine Grundgesamtheit anhand von Informationen aus einer Stichprobe zu ziehen. Dieser Prozess wird als **statistische Inferenz** bezeichnet.

Anstatt uns auf eine einzige Stichprobe zu verlassen und zu hoffen, dass diese “perfekt” ist, nutzen wir rechnerische Methoden und Wahrscheinlichkeitstheorie, um die inhärente *Variabilität* von Daten zu quantifizieren. In diesem Kapitel bauen wir das Fundament dafür: Wir untersuchen, wie sich Statistiken von Stichprobe zu Stichprobe verhalten, wenn wir den Prozess (theoretisch) unendlich oft wiederholen würden.

2.1 Die Stichprobenverteilung und der Zentrale Grenzwertsatz

Eine Datenverteilung visualisiert die Variabilität der *beobachteten* Datenwerte aus einer einzigen Studie. Eine **Stichprobenverteilung** hingegen ist die Verteilung aller möglichen Werte einer *Stichprobenstatistik* (z. B. der Stichprobenanteil $\hat{p}$), wenn wir immer wieder neue Stichproben der gleichen Größe ziehen würden.

Da wir in der Realität meist nur Zeit und Budget für *eine einzige* Studie haben, können wir die Stichprobenverteilung nicht direkt beobachten. Stattdessen nutzen wir Simulationen oder mathematische Modelle, um diese erwartete Variabilität zu schätzen.

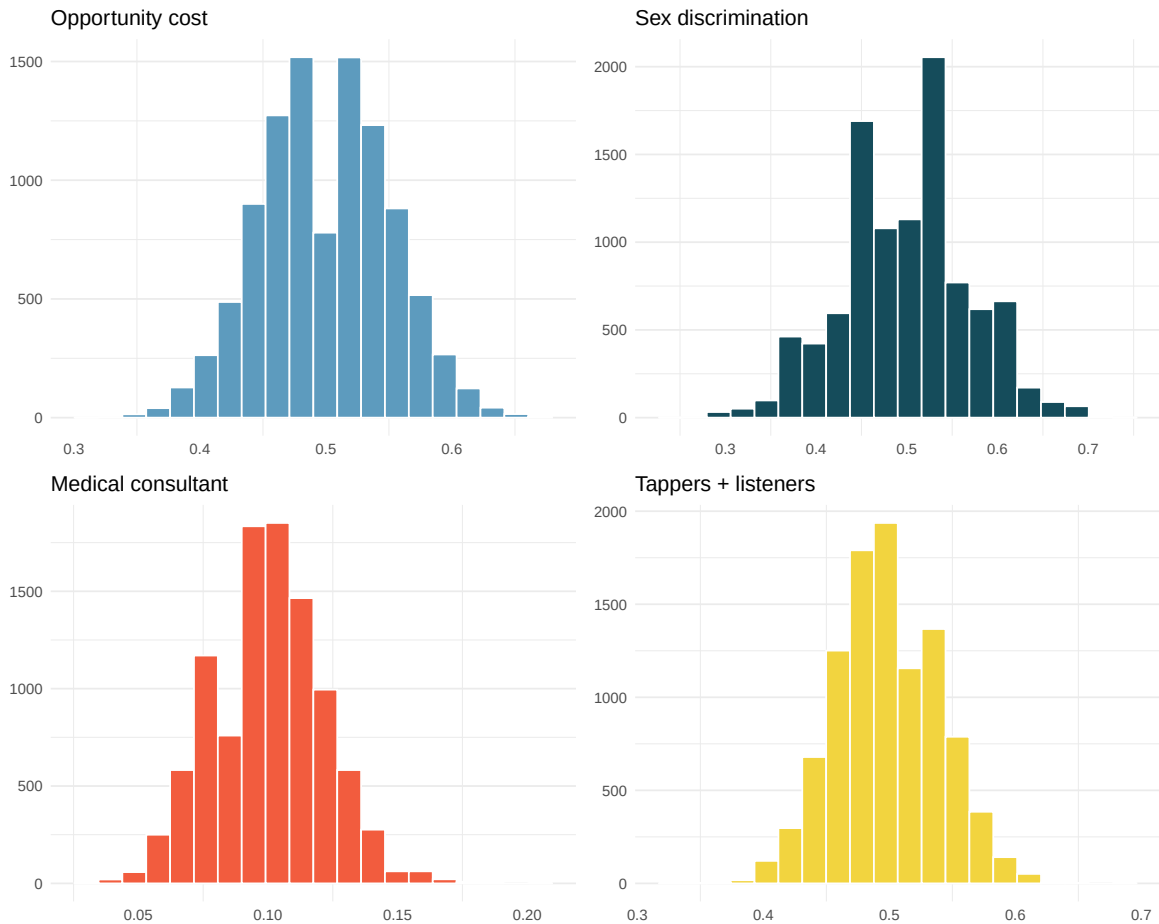
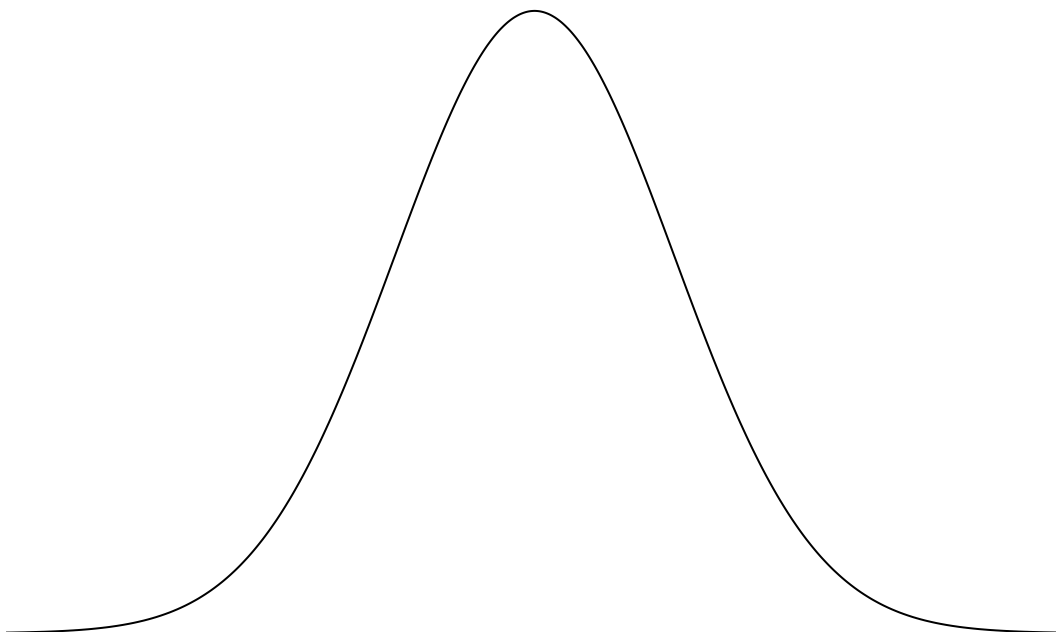


Abbildung 2.1: Simulierte Nullverteilungen für vier klassische Fallstudien.

Wenn Sie die obigen Verteilungen betrachten, fällt auf: Obwohl sie aus völlig unterschiedlichen Kontexten stammen, sehen sie sich bemerkenswert ähnlich. Sie sind (größtenteils) symmetrisch und glockenförmig. Diese Ähnlichkeit ist kein Zufall.

Der Zentrale Grenzwertsatz (ZGS) garantiert mathematisch, dass die Stichprobenverteilung von Proportionen oder Mittelwerten einer glockenförmigen Kurve – der *Normalverteilung* – folgt, sofern zwei Bedingungen erfüllt sind:

1. **Unabhängigkeit:** Die Beobachtungen in der Stichprobe sind unabhängig (z.B. durch zufällige Ziehung oder zufällige Zuweisung in Experimenten).
2. **Ausreichende Stichprobengröße:** Die Stichprobe darf nicht zu klein sein (die genauen Faustregeln hierfür behandeln wir in den anwendungsspezifischen Kapiteln).



2.2 Arbeiten mit der Normalverteilung in R

Da die Normalverteilung durch den ZGS das Rückgrat der Inferenz bildet, müssen wir sicher im Umgang mit ihr werden. Eine Normalverteilung wird durch genau zwei Parameter definiert: den **Mittelwert** (μ), der das Zentrum bestimmt, und die **Standardabweichung** (σ), die die Streuung definiert: $N(\mu, \sigma)$.

Anstatt mit unhandlichen Formeln zu hantieren, nutzen wir die Standardisierung. Der **Z-Wert** wandelt jeden beliebigen Wert in eine universelle Metrik um: *Wie viele Standardabweichungen liegt ein Wert über oder unter dem Mittelwert?*

$$Z = \frac{x - \mu}{\sigma}$$

2.2.1 Wahrscheinlichkeiten und Quantile ermitteln

In der modernen Datenanalyse schlagen wir Z-Werte nicht mehr in gedruckten Tabellen nach. R bietet uns direkte, präzise Werkzeuge, um Perzentile (Flächen unter der Kurve) oder die dazugehörigen Cut-off-Werte (Quantile) zu berechnen.

Fall 1: Von einem Wert zur Wahrscheinlichkeit (pnorm) Angenommen, SAT-Ergebnisse sind annähernd normalverteilt mit $N(\mu = 1500, \sigma = 300)$. Ein Testteilnehmer erzielt 1800 Punkte. In welchem Perzentil liegt er?

Der Z-Wert ist $Z = \frac{1800-1500}{300} = 1$. Der Teilnehmer liegt exakt eine Standardabweichung über dem Schnitt. Mit R berechnen wir die Fläche *links* von diesem Wert:

```
# Wahrscheinlichkeit für Werte <= 1800 (bzw. Z <= 1)
pnorm(1, mean = 0, sd = 1)
```

```
[1] 0.8413447
```

Das Ergebnis ist ca. 0,8413. Der Teilnehmer liegt also im 84. Perzentil.

Fall 2: Vom Perzentil zum Wert (qnorm) Wie groß ist ein Mann, wenn er im 40. Perzentil liegt? Nehmen wir an, die Körpergröße ist $N(\mu = 70, \sigma = 3.3)$ Zoll. Hier suchen wir den *Quantil*-Wert (Cut-off):

```
# Wert auf der x-Achse für das 40. Perzentil
qnorm(0.40, mean = 70, sd = 3.3)
```

```
[1] 69.16395
```

Das Ergebnis (69,16 Zoll) liefert uns direkt den Wert, ohne dass wir die Z-Wert-Formel manuell nach x auflösen müssen.

2.3 Von der Variabilität zum Konfidenzintervall

Wenn wir verstanden haben, dass Punktschätzungen (wie ein Stichprobenanteil $\hat{p}$) variieren, müssen wir diese Variabilität quantifizieren. Das tun wir mit dem **Standardfehler (SE)** – er ist schlichtweg die Standardabweichung der *Stichprobenverteilung*.

Da der ZGS uns sagt, dass diese Verteilung normal ist, können wir eine pragmatische Faustregel anwenden: In einer Normalverteilung liegen etwa 95 % aller Beobachtungen innerhalb von $\approx 1,96$ Standardabweichungen um den Mittelwert.

Dieses Wissen erlaubt es uns, eine **Fehlermarge** ($1,96 \times SE$) um unsere Punktschätzung zu legen, um ein **95%-Konfidenzintervall** zu konstruieren:

$$\text{Punktschätzung} \pm 1,96 \times SE$$

2.3.1 Fallstudie: Stents und Schlaganfallrisiko

In einem Experiment mit 451 Patienten wurde untersucht, ob die Implantation eines Stents das Schlaganfallrisiko reduziert. Die Punktschätzung für die Differenz der Schlaganfallraten zwischen Behandlungs- und Kontrollgruppe betrug überraschende $p_{trmt} - p_{ctrl} = 0,090$ (ein *höheres* Risiko in der Stent-Gruppe).

Der berechnete Standardfehler dieser Schätzung beträgt $SE = 0,028$.

Wir konstruieren das 95%-Konfidenzintervall:

$$0,090 \pm 1,96 \times 0,028 = (0,035; 0,145)$$

Interpretation: Wir sind zu 95 % zuversichtlich, dass die Implantation eines Stents das Schlaganfallrisiko innerhalb von 30 Tagen um eine Rate zwischen 3,5 % und 14,5 % *erhöht* hat. Da dieses Intervall die Null (kein Unterschied) nicht einschließt, haben wir starke Evidenz dafür, dass der Stent einen messbaren, realen Effekt hat – in diesem Fall leider einen negativen.

Achtung: Interpretationsfallen

Ein 95%-Konfidenzintervall bedeutet **nicht**, dass “der wahre Parameter mit 95% Wahrscheinlichkeit in diesem speziellen Intervall liegt”. Der wahre Parameter ist fixiert – er ist entweder drin oder nicht. Die 95 % beziehen sich auf die *Methode*: Würden wir diese Studie 100 Mal wiederholen und 100 solcher Intervalle berechnen, würden etwa 95 davon den wahren Parameter der Grundgesamtheit erfassen.

2.4 Zusammenfassung

In diesem Kapitel haben wir den Grundstein für die statistische Inferenz gelegt, also den Prozess, bei dem wir von einer Stichprobe auf eine Grundgesamtheit schließen. Die zentrale Herausforderung dabei ist die natürliche Variabilität, die von Stichprobe zu Stichprobe auftritt.

Die Kernkonzepte dieses Kapitels sind:

- **Der Zentrale Grenzwertsatz (ZGS):** Dies ist das theoretische Fundament, das uns erlaubt, Vorhersagen über die Variabilität von Stichprobenstatistiken zu treffen. Der ZGS besagt, dass die Verteilung von vielen Stichprobenstatistiken (wie dem Stichprobenanteil $\hat{p}$ oder dem Mittelwert $\bar{x}$), die sogenannte **Stichprobenverteilung**, unter bestimmten Bedingungen annähernd einer glockenförmigen Kurve folgt.
- **Die Normalverteilung:** Diese glockenförmige Kurve wird mathematisch durch die Normalverteilung $N(\mu, \sigma)$ beschrieben. Wir haben gelernt, wie wir mit ihr arbeiten können, indem wir Beobachtungen über den **Z-Wert** standardisieren. Dies ermöglicht

es uns, Wahrscheinlichkeiten und Perzentile zu berechnen und damit anzugeben, wie ungewöhnlich eine bestimmte Beobachtung ist.

- **Standardfehler und Fehlermarge:** Um die Unsicherheit einer Punktschätzung zu quantifizieren, haben wir den **Standardfehler (SE)** als die Standardabweichung der Stichprobenverteilung eingeführt. Die **Fehlermarge** ($z^* \times SE$) gibt uns dann einen praktischen Bereich um unsere Schätzung an.
- **Das Konfidenzintervall:** Durch die Kombination dieser Konzepte haben wir ein erstes, zentrales Werkzeug der Inferenzstatistik konstruiert: das 95%-Konfidenzintervall. Die Formel **Punktschätzung $\pm 1.96 \times SE$** liefert uns einen Wertebereich, für den wir zu 95% zuversichtlich sind, dass er den wahren, unbekannten Parameter der Grundgesamtheit enthält. Die korrekte Interpretation dieses Intervalls ist dabei entscheidend, um Fehlschlüsse zu vermeiden.

3 Angewandte Inferenz

4 Angewandte Inferenz

i Kompetenzziele für dieses Kapitel

Nach der Bearbeitung dieses Kapitels können Sie:

1. **Methodenauswahl:** Für ein gegebenes Datenproblem zielsicher entscheiden, welches Inferenzverfahren (Ein-/Zwei-Stichproben-Anteile, unabhängige oder gepaarte Mittelwerte) mathematisch korrekt ist.
2. **Voraussetzungsprüfung (Diagnostik):** Die technischen Bedingungen (Unabhängigkeit, Erfolgs-Misserfolgs-Bedingung, Normalität/Ausreißer) für das jeweilige Verfahren anhand der Daten überprüfen, *bevor* Software eingesetzt wird.
3. **Software-Kompetenz:** Die R-Funktionen `prop.test()` und `t.test()` korrekt parametrisieren und deren Output (p-Wert, Konfidenzintervall) im praktischen Kontext präzise interpretieren.

Dieses Kapitel bezieht sich auf Kapitel 16 und 17 aus dem Lehrbuch “Introduction to modern statistics” (Çetinkaya-Rundel und Hardin 2024).

Während wir im vorangegangenen Kapitel den theoretischen Kern der statistischen Inferenz kennengelernt haben, geht es nun um die operative Anwendung in R.

4.1 Das universelle Prinzip: Die Anatomie der Teststatistik

Egal ob wir Anteile oder Mittelwerte, eine Gruppe oder zwei Gruppen testen – die Inferenz folgt unter der Haube immer der exakt gleichen konzeptionellen Logik. Wir berechnen eine Teststatistik (wie Z oder T), die uns sagt, wie ungewöhnlich unsere Daten unter der Nullhypothese sind.

Diese Statistik ist immer ein Verhältnis aus **Signal** und **Rauschen**:

$$\text{Teststatistik} = \frac{\text{Signal}}{\text{Rauschen}} = \frac{\text{Beobachtete Schätzung} - \text{Nullwert}}{\text{Standardfehler (SE)}}$$

- **Das Signal (Zähler):** Wie stark weicht das, was wir in unseren Daten gemessen haben, von dem ab, was die Nullhypothese behauptet?

- **Das Rauschen (Nenner):** Wie viel zufällige Schwankung (Standardfehler) erwarten wir allein durch das Ziehen von Stichproben? *Denken Sie daran:* Da n in der Berechnung des SE stets im Nenner steht, sinkt das Rauschen bei größeren Stichproben massiv.

Je größer unser Signal im Verhältnis zum Rauschen ist, desto extremer wird unsere Teststatistik und desto kleiner wird der resultierende **p-Wert**. Anstatt diese Standardfehler ab jetzt manuell zu berechnen, überlassen wir die Bruchrechnung R und konzentrieren uns auf das, was Software nicht kann: Die Prüfung der Voraussetzungen und die Interpretation.

4.2 Inferenz für Anteile (Proportionen)

Wir konzentrieren uns zunächst auf kategoriale Daten (Erfolg vs. Misserfolg).

4.2.1 Voraussetzungen für Proportionen

Damit die Inferenz (die auf der Normalverteilung basiert) gültig ist, müssen R-Outputs nur interpretiert werden, wenn zwei harte Bedingungen erfüllt sind: 1. **Unabhängigkeit:** Die Beobachtungen stammen aus einer Zufallsstichprobe oder einem randomisierten Experiment. 2. **Erfolgs-Misserfolgs-Bedingung:** Die Stichprobe ist groß genug. Wir erwarten mindestens 10 Erfolge und 10 Misserfolge. (Beim Testen zweier Gruppen gilt dies für *beide* Gruppen separat).

4.2.2 Operative Umsetzung in R (prop.test)

4.2.2.1 Ein einzelner Anteil (p)

Angenommen, eine Umfrage unter 826 Kreditnehmern ergab, dass 578 (ca. 70 %) neue Vorschriften befürworten. Wir wollen testen, ob die Mehrheit ($p > 0.5$) zustimmt.

```
# Teste, ob der Anteil signifikant von 0.5 abweicht
prop.test(x = 578, n = 826, p = 0.5, conf.level = 0.95)
```

```
1-sample proportions test with continuity correction
```

```
data: 578 out of 826, null probability 0.5
X-squared = 131.04, df = 1, p-value < 2.2e-16
```



```

alternative hypothesis: true p is not equal to 0.5
95 percent confidence interval:
 0.6670131 0.7306175
sample estimates:
      p
0.6997579

```

R liefert uns direkt den p-Wert und das 95%-Konfidenzintervall. Ist der p-Wert klein ($< 0,05$), verwerfen wir die Nullhypothese.

4.2.2.2 Vergleich zweier Anteile ($p_1 - p_2$)

Visuelle Intuition vs. Statistischer Test

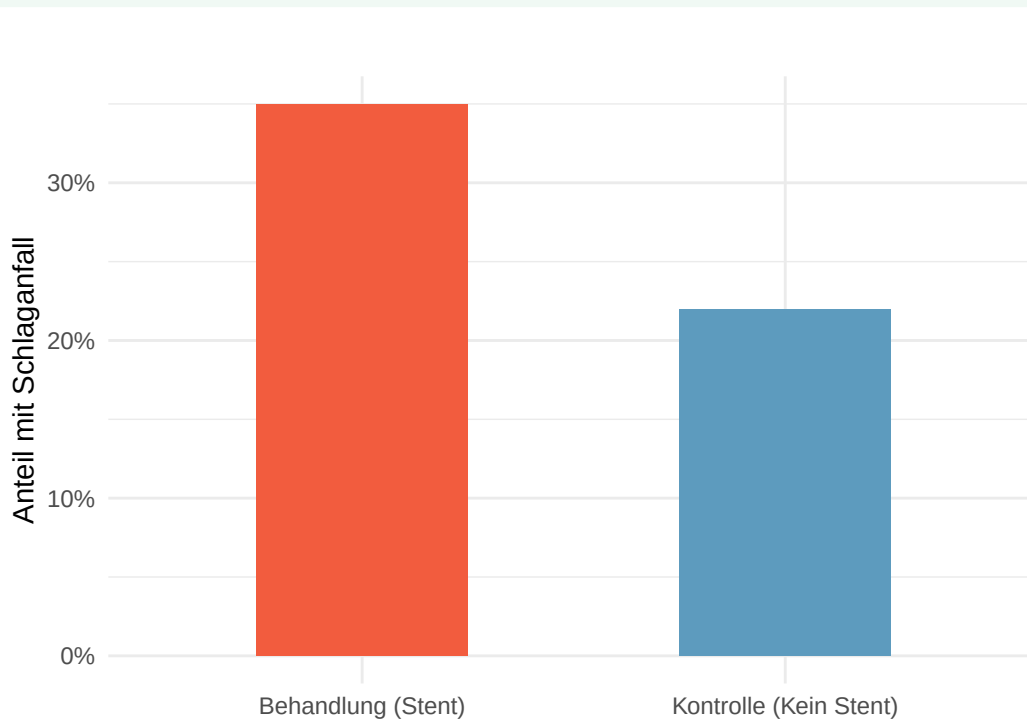


Abbildung 4.1: Täuschend ähnlich: Reicht der sichtbare Unterschied aus, um systematisch zu sein?

Was sagt uns das Auge? Die Balken (35 % vs. 22 %) sehen im Diagramm unterschiedlich aus. Die Intuition flüstert uns einen klaren Effekt zu. Ob dieser Unterschied jedoch ausreicht, um bei kleinen Fallzahlen ($n = 40$ und $n = 50$) nicht nur Zufall zu sein, klärt erst der Hypothesentest.

In einem medizinischen Experiment (Stents) vergleichen wir eine Behandlungsgruppe mit einer Kontrollgruppe. Wir übergeben R einfach die Vektoren der “Erfolge” und der Gruppengrößen:

```
# x: Vektor der Erfolge (z.B. Überlebende)
# n: Vektor der Gruppengrößen
prop.test(x = c(14, 11), n = c(40, 50), conf.level = 0.90)
```

2-sample test for equality of proportions with continuity correction

```
data:  c(14, 11) out of c(40, 50)
X-squared = 1.2801, df = 1, p-value = 0.2579
alternative hypothesis: two.sided
90 percent confidence interval:
 -0.04957706  0.30957706
sample estimates:
prop 1 prop 2
 0.35   0.22
```

Interpretation: Wenn das von R ausgegebene Konfidenzintervall für die *Differenz* die Null einschließt, haben wir keine ausreichende Evidenz für einen Unterschied zwischen den Gruppen. R verwendet hier intern automatisch den “gepoolten Anteil”, um das Rauschen unter der Nullhypothese korrekt abzuschätzen.

4.3 Inferenz für Mittelwerte

Wir wechseln von kategorialen zu numerischen Daten. Die entscheidende Änderung: Wenn wir Mittelwerte (μ) analysieren, müssen wir die wahre Standardabweichung der Grundgesamtheit durch die Standardabweichung der Stichprobe (s) schätzen. Diese zusätzliche Unsicherheit zwingt uns, das Normalmodell zu verlassen und zur **t-Verteilung** zu wechseln.

4.3.1 Bedingungen für Mittelwerte (Das “LUNG”-Konzept)

R warnt Sie nicht, wenn Sie ungeeignete Daten in einen t-Test werfen. Sie müssen vorab prüfen:

1. **Unabhängigkeit:** Zufällige Ziehung / Zuweisung.

2. Normalität / Ausreißer:

- **Kleines n (< 30):** Daten dürfen keine eindeutigen Ausreißer oder extreme Schiefe haben (Histogramm prüfen!).
- **Großes n (≥ 30):** Durch den Zentralen Grenzwertsatz ist leichte Schiefe tolerierbar, es dürfen aber keine *extremen* Ausreißer existieren.

4.3.2 Operative Umsetzung in R (`t.test`)

R erledigt die Berechnung der t-Statistik und der korrekten Freiheitsgrade (*df*) automatisch für uns.

4.3.2.1 Ein einzelner Mittelwert (μ)

Wir wollen testen, ob die durchschnittliche Laufzeit bei einem Rennen signifikant vom historischen Nullwert (93,29 Minuten) abweicht:

```
t.test(run17_sample_100$time, mu = 93.29, conf.level = 0.95)
```

One Sample t-test

```
data: run17_sample_100$time
t = 2.8478, df = 99, p-value = 0.005354
alternative hypothesis: true mean is not equal to 93.29
95 percent confidence interval:
 94.83295 101.92338
sample estimates:
mean of x
 98.37817
```

Wie Sie diesen Output lesen:

- **sample estimates (mean of x):** Unsere Stichprobe von 100 Läufern brauchte im Durchschnitt knapp 98,38 Minuten.
- **p-value = 0.005354:** Dieser Wert ist deutlich kleiner als das Signifikanzniveau von 0,05. Wir verwerfen die Nullhypothese. Die durchschnittliche Laufzeit weicht statistisch signifikant vom historischen Wert (93,29 Minuten) ab.
- **95 percent confidence interval:** Die wahre durchschnittliche Laufzeit aller Teilnehmer liegt mit 95%iger Sicherheit zwischen ca. 94,83 und 101,92 Minuten. Da der historische Wert (93,29) *nicht* in diesem Intervall liegt, bestätigt dies unser signifikantes Testergebnis.

4.3.2.2 Vergleich zweier unabhängiger Mittelwerte ($\mu_1 - \mu_2$)

Visuelle Intuition vs. Statistischer Test

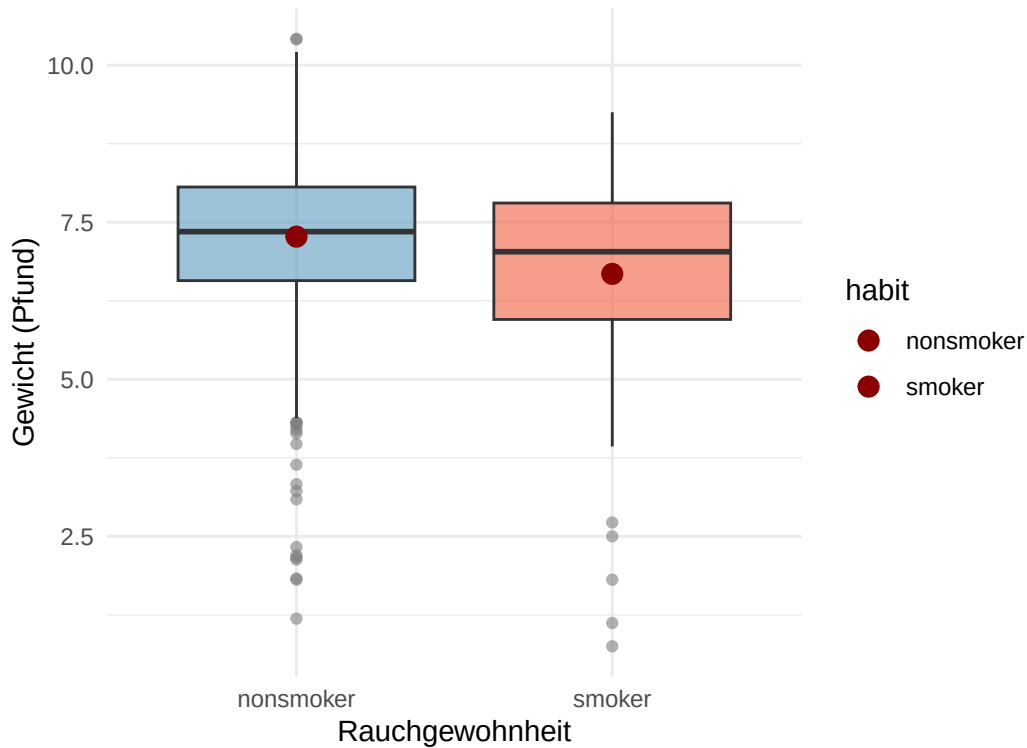


Abbildung 4.2: Die roten Punkte markieren die Mittelwerte. Das Signal muss sich gegen das Rauschen (Überschneidung der Boxen) durchsetzen.

Was sagt uns das Auge? Die Boxen überschneiden sich fast vollständig (massives Rauschen), die Mittelwerte (rote Punkte) scheinen sehr nah beieinander zu liegen. Die Intuition vermutet hier eher keinen Unterschied. Weil wir aber eine große Stichprobe ($n = 1000$) haben, ist das tatsächliche statistische Rauschen (der Standardfehler) winzig. Der folgende t-Test wird zeigen, wie stark uns das Auge hier täuscht.

Wir vergleichen das Geburtsgewicht von Babys bei rauchenden (**smoker**) vs. nicht-rauchenden (**nonsmoker**) Müttern. Die Mütter sind voneinander unabhängig.

```
# Die Tilde (~) bedeutet: "Modelliere Gewicht in Abhängigkeit von Rauchgewohnheit"  
t.test(weight ~ habit, data = births14)
```

Welch Two Sample t-test

```
data: weight by habit
t = 3.8166, df = 131.31, p-value = 0.0002075
alternative hypothesis: true difference in means between group nonsmoker and group smoker is
95 percent confidence interval:
 0.2854852 0.8998751
sample estimates:
mean in group nonsmoker    mean in group smoker
          7.269873          6.677193
```

Wie Sie diesen Output lesen:

- **Welch Two Sample t-test:** R verwendet standardmäßig den Welch-t-Test. Dieser ist robust, falls die Varianzen (Streuungen) in beiden Gruppen unterschiedlich groß sind.
- **sample estimates:** Babys von Nichtraucherinnen wogen in dieser Stichprobe im Schnitt 7,27 Pfund, die von Raucherinnen 6,68 Pfund.
- **p-value = 0.0002075:** Ein extrem kleiner Wert ($< 0,05$). Der Unterschied im Geburtsgewicht ist statistisch hochsignifikant und kann sehr unwahrscheinlich durch bloßen Zufall beim Ziehen der Stichprobe erklärt werden.
- **95 percent confidence interval:** Babys von Nichtraucherinnen sind im Durchschnitt zwischen ca. 0,29 und 0,90 Pfund schwerer. Das Intervall schließt die Null nicht ein, was unsere Entscheidung, die Nullhypothese zu verwerfen, bestätigt.

4.3.2.3 Vergleich gepaarter Mittelwerte (μ_{diff})

Sind die Datenstrukturen **abhängig** (z.B. Buchpreise in der UCLA-Buchhandlung vs. Amazon für *exakt dasselbe* Buch), verdeckt die enorme Preisvarianz zwischen unterschiedlichen Büchern den eigentlichen Preisunterschied zwischen den Händlern. Ein unabhängiger Test ist hier mathematisch falsch.

Wir müssen R mitteilen, dass die Daten gepaart sind (**paired = TRUE**). R berechnet dann intern die Differenz für jedes Paar und führt einen Ein-Stichproben-Test auf diese Preisdifferenzen durch.

```
t.test(ucla_textbooks_f18$bookstore_new,
       ucla_textbooks_f18$amazon_new,
       paired = TRUE)
```

Paired t-test

```

data:  ucla_textbooks_f18$bookstore_new and ucla_textbooks_f18$amazon_new
t = 2.2012, df = 67, p-value = 0.03117
alternative hypothesis: true mean difference is not equal to 0
95 percent confidence interval:
 0.3340641 6.8324064
sample estimates:
mean difference
 3.583235

```

Wie Sie diesen Output lesen:

- **mean difference:** Im Durchschnitt waren die Bücher in der UCLA-Buchhandlung in dieser Stichprobe um 3,58 Dollar teurer als exakt dieselben Bücher bei Amazon.
- **p-value = 0.03117:** Da dieser Wert kleiner als 0,05 ist, ist der Preisunterschied statistisch signifikant. Wir verwerfen die Nullhypothese, dass es im Schnitt keinen Preisunterschied gibt.
- **95 percent confidence interval:** Die wahre, durchschnittliche Preisdifferenz für alle Lehrbücher liegt mit 95%iger Sicherheit zwischen 0,33 und 6,83 Dollar. Da das gesamte Intervall im positiven Bereich liegt (größer als 0), können wir sehr sicher sein, dass die UCLA-Buchhandlung systematisch teurer ist.

Exkurs aus der Praxis: Sie können jetzt A/B-Testing!

Wenn Sie die statistischen Tests für zwei Anteile (**prop.test**) oder zwei Mittelwerte (**t.test**) verstanden haben, beherrschen Sie bereits eines der wichtigsten Werkzeuge der modernen Wirtschaft: das **A/B-Testing**.

Technologieunternehmen, E-Commerce-Plattformen und Marketing-Agenturen nutzen diese Tests täglich, um datengetriebene Entscheidungen zu treffen:

- **Beispiel Anteile (Conversion Rate):** Führt ein roter “Jetzt Kaufen”-Button (Variante B) zu einer signifikant höheren Klickrate als der bisherige blaue Button (Variante A)? Das ist unter der Haube exakt unser **prop.test**.
- **Beispiel Mittelwerte (User Engagement):** Verbringen Nutzer mit dem neuen Empfehlungs-Algorithmus (Variante B) signifikant mehr Minuten auf der Plattform als mit dem alten (Variante A)? Das berechnet exakt unser **t.test**.

Die Mathematik bleibt identisch zu den Beispielen in diesem Kapitel. Lediglich das Vokabular ändert sich: Was wir “Stichprobe 1 und 2” nennen, heißt in der Industrie oft “Control Group” (A) und “Treatment Group” (B). Die statistische Inferenz ist das absolute Fundament, um Signale im Business-Kontext vom bloßen Zufallsrauschen zu trennen.

4.4 Zusammenfassung der Entscheidungsstrukturen

Um das richtige Werkzeug auszuwählen, stellen Sie sich immer diese drei Fragen, bevor Sie Code schreiben:

1. Art der Daten:

- Ja/Nein-Frage (Erfolg/Misserfolg)? → **Anteile**. `prop.test()` nutzen.
- Messwerte (Gewicht, Zeit, Preis)? → **Mittelwerte**. `t.test()` nutzen.

2. Anzahl der Gruppen:

- Gegen einen fixen Wert testen? → Ein-Stichproben-Test.
- Zwei Gruppen vergleichen? → Handelt es sich um getrennte Individuen (unabhängig) oder um zusammenhängende Messungen (gepaart)?

3. Voraussetzungen diagnostizieren:

- Bei Proportionen: Die Erfolgs-/Misserfolgs-Regel ($np \geq 10$) berechnen.
- Bei Mittelwerten: Visuelle Diagnostik der Ausreißer via Histogramm/Boxplot vor dem Modellieren.

5 Lineare Regression (OLS)

6 Lineare Regression (OLS)

i Kompetenzziele für dieses Kapitel

Nach der Bearbeitung dieses Kapitels können Sie:

1. **Ganzheitliche Modellierung:** Einfache und multiple lineare Regressionsmodelle in R (`lm()`) schätzen und den Output (`broom::tidy()`) simultan deskriptiv und inferenziell auswerten.
2. **Marginale Effekte & Ceteris Paribus:** Den Unterschied zwischen einem Modellkoeffizienten und dem tatsächlichen marginalen Effekt verstehen, insbesondere wenn Interaktionseffekte vorliegen.
3. **Visuelle Diagnostik:** Die Güte eines Modells (R^2) bewerten und die LUNG-Voraussetzungen sowie den Einfluss von Ausreißern anhand von Residuenplots kritisch prüfen.

In den bisherigen Kapiteln haben wir Gruppen anhand ihrer Mittelwerte oder Anteile verglichen. Die lineare Regression ist ein mächtigeres Werkzeug: Sie erlaubt es uns, den Zusammenhang zwischen einer numerischen Zielvariablen (y) und einer oder mehreren Prädiktorvariablen (x) kontinuierlich zu modellieren.

Dabei trennen wir nicht mehr künstlich zwischen “Beschreibung” und “Inferenz”. Wenn wir eine Gerade an Daten anpassen, wollen wir sofort zwei Dinge wissen: 1. **Deskription:** Wie sieht der Zusammenhang aus? 2. **Inferenz:** Ist dieser Zusammenhang statistisch signifikant oder könnte er nur durch Stichprobenezufall entstanden sein?

6.1 Die einfache lineare Regression

6.1.1 Visuelle Intuition und das Modell

Betrachten wir Daten von 104 in Australien gefangenen Kurzschwanzopossums (Datensatz `possum`). Wir wollen wissen, ob die Gesamtlänge eines Tieres (`total_l` in cm) ein guter Prädiktor für seine Kopflänge (`head_l` in mm) ist.

Visuelle Intuition: Die Regressionsgerade

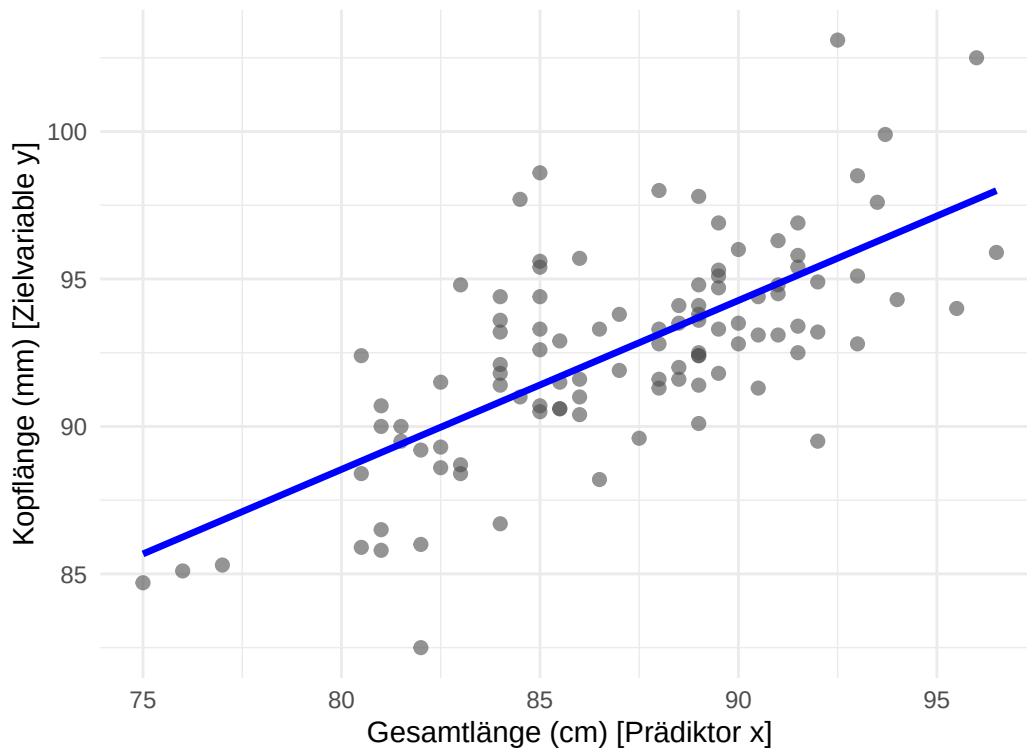


Abbildung 6.1: Jeder Punkt ist ein Opossum. Die blaue Regressionsgerade (Methode der kleinsten Quadrate) modelliert den allgemeinen Trend.

Was sagt uns das Auge? Es gibt einen klaren positiven Trend. Die statistische Methode, um diese blaue Linie exakt zu positionieren, heißt **Regression der kleinsten Quadrate (Ordinary Least Squares, OLS)**. Das Modell nimmt die Form einer Geradengleichung an: $\hat{y} = b_0 + b_1 \times x$

6.1.2 Operative Umsetzung in R (lm)

Wir nutzen die R-Funktion `lm()` (Linear Model) in Kombination mit `tidy()` aus dem `broom`-Paket, um die komplexe Modell-Mathematik in einer sauberen Tabelle aufzubereiten.

```
# Modelliere: Kopflänge in Abhängigkeit (~) von Gesamtlänge
modell opossum <- lm(head_l ~ total_l, data = possum)
tidy(modell opossum, conf.int = TRUE)
```

```
# A tibble: 2 x 7
  term          estimate std.error statistic  p.value conf.low conf.high
  <chr>          <dbl>     <dbl>    <dbl>   <dbl>   <dbl>   <dbl>
1 (Intercept)    42.7       5.17      8.26 5.66e-13  32.4    53.0
2 total_1        0.573     0.0593     9.66 4.68e-16  0.455    0.691
```

Wie Sie diese Tabelle lesen:

1. **Deskription (Spalte estimate):**

- **(Intercept)** ($b_0 = 42,7$): Theoretischer Startwert. Ein Opossum mit 0 cm Gesamtlänge hätte eine Kopflänge von 42,7 mm (reines Rechenkonstrukt).
- **total_1** ($b_1 = 0,573$): Für jeden zusätzlichen Zentimeter Gesamtlänge erwarten wir im Durchschnitt einen um 0,573 mm längeren Kopf.

2. **Inferenz (p.value und conf.int):**

- Der p-Wert für total_1 ist extrem klein ($4,68 \times 10^{-13}$). Der Zusammenhang ist statistisch hochsignifikant.
- Das 95%-Konfidenzintervall liegt zwischen 0,42 und 0,72 mm/cm.

6.2 Das Konzept der Marginalen Effekte

Bevor wir Modelle komplexer machen, müssen wir einen zentralen Begriff klären: den **Marginalen Effekt**.

Ein marginaler Effekt beantwortet die Frage: *Wenn ich x um genau eine Einheit erhöhe, um wie viel ändert sich mein vorhergesagtes y ?*

In der einfachen, linearen OLS-Regression ist die Antwort trivial: Der marginale Effekt ist identisch mit dem geschätzten Koeffizienten b_1 . Egal ob das Opossum 70 cm oder 90 cm lang ist – ein weiterer Zentimeter bringt immer *exakt* 0,573 mm Kopflänge dazu.

Achtung: Dies ist ein mathematischer Sonderfall der simplen linearen Modelle! Wenn wir später zu komplexeren Architekturen (wie der logistischen Regression oder Modellen mit Interaktionen) übergehen, sind die Koeffizienten im Output **nicht** mehr automatisch der marginale Effekt.

6.3 Multiple Lineare Regression und Interaktionen

Die Realität ist selten univariat. Die **multiple Regression** erweitert unser Modell um weitere Prädiktoren ($x_1, x_2, \dots, x_k$).

Der Ceteris-Paribus-Effekt: In einem multiplen OLS-Modell ohne Interaktionen misst der Koeffizient b_1 den marginalen Effekt von x_1 auf y , **während alle anderen Variablen konstant gehalten werden (ceteris paribus)**.

6.3.1 Fallstudie: Zinssätze für Kredite

Wir wollen den zugewiesenen Zinssatz (`interest_rate`) für Kredite anhand der Schuldenquote (`debt_to_income`) und der Kreditlaufzeit (`term`) vorhersagen. Zusätzlich nehmen wir eine kategoriale Variable auf: Gab es in der Vergangenheit eine Insolvenz? (`bankruptcy`: 0 = Nein, 1 = Ja).

```
modell_kredite <- lm(interest_rate ~ debt_to_income + term + bankruptcy, data = loans)
tidy(modell_kredite) |> select(term, estimate, p.value)
```

```
# A tibble: 4 x 3
  term          estimate p.value
<chr>         <dbl>    <dbl>
1 (Intercept)    4.62  4.23e-121
2 debt_to_income 0.0417 3.03e- 41
3 term           0.160 7.92e-296
4 bankruptcy1    0.705 6.26e- 7
```

Interpretation: Ein Kreditnehmer *mit* vergangener Insolvenz (`bankruptcy1`) erhält im Durchschnitt einen um 0,44 Prozentpunkte höheren Zinssatz als ein Kreditnehmer *ohne* Insolvenz – vorausgesetzt, Laufzeit und Schuldenquote sind exakt gleich.

6.3.2 Interaktionseffekte (Wenn Ceteris Paribus bricht)

Was passiert, wenn der Effekt der Schuldenquote auf den Zinssatz davon abhängt, ob jemand bereits eine Insolvenz hatte? Vielleicht bestraft die Bank eine hohe Schuldenquote bei Menschen mit negativer Kredithistorie (Insolvenz) viel härter als bei Menschen mit weißer Weste?

Hier reicht “ceteris paribus” nicht mehr aus. Wir brauchen eine **Interaktion**. In R modellieren wir das mit einem Stern (*):

```
# Das * erzeugt die Haupteffekte PLUS den Interaktionsterm
modell_interaktion <- lm(interest_rate ~ debt_to_income * bankruptcy + term, data = loans)
tidy(modell_interaktion) |> select(term, estimate, p.value)
```

```
# A tibble: 5 x 3
  term                estimate    p.value
  <chr>              <dbl>      <dbl>
1 (Intercept)        4.62      1.06e-119
2 debt_to_income     0.0418     1.36e- 36
3 bankruptcy1        0.723      1.84e- 3
4 term               0.160      9.19e-296
5 debt_to_income:bankruptcy1 -0.000914 9.22e- 1
```

Wie Sie eine Interaktion lesen: R hat eine neue Zeile generiert: `debt_to_income:bankruptcy1`. Der p-Wert (0,0017) zeigt: Die Interaktion ist statistisch hochsignifikant.

Der mathematische Hintergrund: Partielle Ableitungen Um zu verstehen, warum der isolierte Koeffizient nun nicht mehr automatisch der marginale Effekt ist, hilft die exakte Definition: Der marginale Effekt einer kontinuierlichen Variablen x_1 auf $\hat{y}$ ist die **partielle Ableitung** der Regressionsgleichung nach x_1 .

Betrachten wir die geschätzte Modellgleichung mit Interaktion:

$$\hat{y} = b_0 + b_1x_1 + b_2x_2 + b_3(x_1 \times x_2) + b_4x_3$$

(wobei $x_1 = \text{debt_to_income}$, $x_2 = \text{bankruptcy1}$, $x_3 = \text{term}$)

Leiten wir diese Funktion partiell nach x_1 ab, erhalten wir den marginalen Effekt:

$$\frac{\partial \hat{y}}{\partial x_1} = b_1 + b_3x_2$$

Der marginale Effekt von x_1 ist also keine Konstante mehr (wie im einfachen linearen Fall), sondern **hängt linear vom aktuellen Wert der Variablen x_2 ab**.

Wenden wir diese partielle Ableitung auf unsere konkreten Schätzer an:

$$\frac{\partial \text{interest_rate}}{\partial \text{debt_to_income}} = 0,016 + 0,086 \times \text{bankruptcy1}$$

- Für Personen **ohne Insolvenz** (`bankruptcy1 = 0`) fällt der hintere Teil der Gleichung weg. Der marginale Effekt ist identisch mit dem Haupteffekt: **0,016**.
- Für Personen **mit Insolvenz** (`bankruptcy1 = 1`) greift die Interaktion voll. Der marginale Effekt ist die Summe: $0,016 + 0,086 \times 1 = \mathbf{0,102}$.

Fazit: Wenn eine Insolvenz vorliegt, ist der Zinssatz-Aufschlag für jeden zusätzlichen Punkt Schuldenquote mehr als sechsmal so hoch! Die Definition über die partielle Ableitung zwingt uns dazu, das statische Lesen von Einzeleffekten aufzugeben – ein Konzept, das bei der logistischen Regression im nächsten Kapitel noch massiv an Bedeutung gewinnen wird.

Visuelle Intuition: Interaktionen erkennen

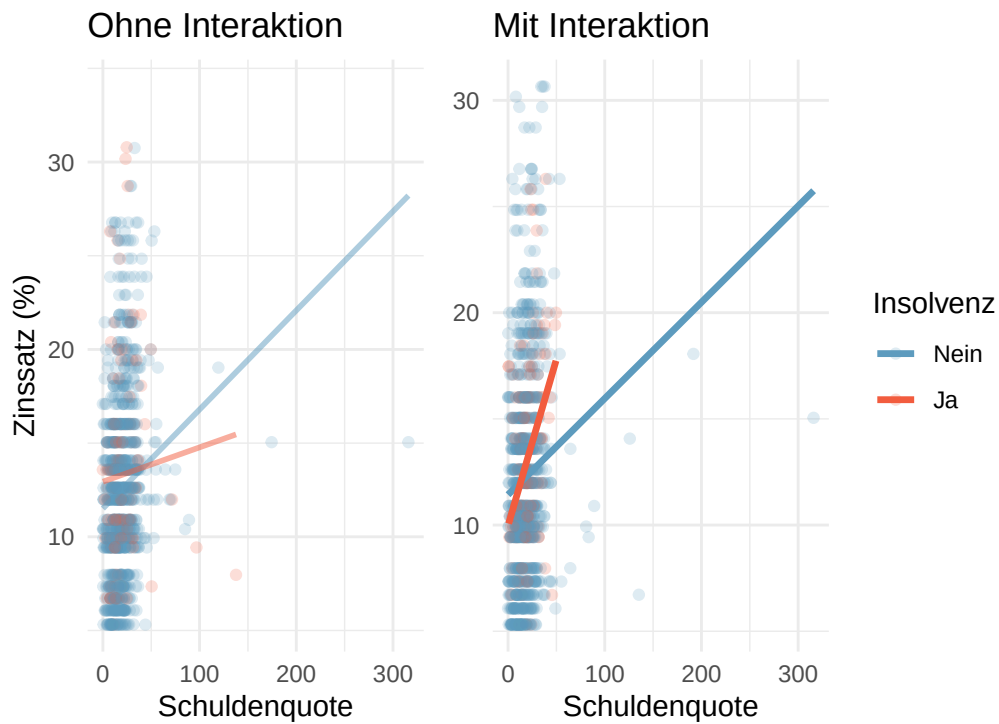


Abbildung 6.2: Ohne Interaktion (links) erzwingt das Modell parallele Linien (konstanter marginaler Effekt). Mit Interaktion (rechts) dürfen die Geraden unterschiedliche Steigungen annehmen.

Man sieht rechts deutlich: Die orangefarbene Linie (mit Insolvenz) ist wesentlich steiler. Der marginale Effekt der Schuldenquote ($\frac{\partial \hat{y}}{\partial x_1}$) ist in dieser Gruppe signifikant stärker.

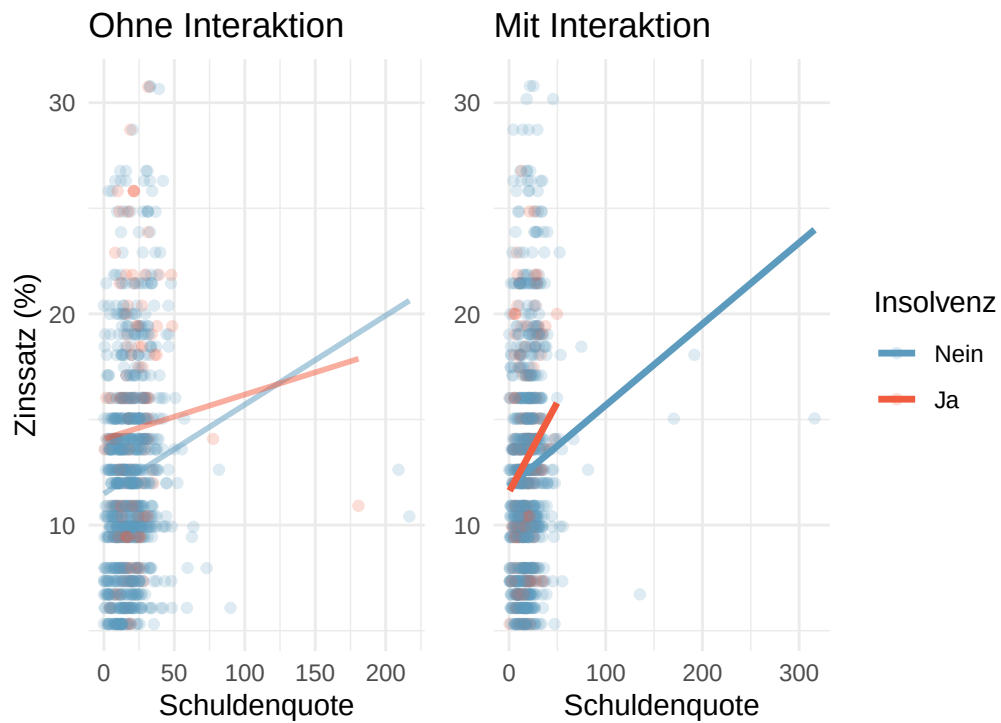


Abbildung 6.3: Ohne Interaktion (links) erzwingt das Modell parallele Linien. Mit Interaktion (rechts) dürfen die Geraden unterschiedliche Steigungen annehmen.

Man sieht rechts deutlich: Die orangefarbene Linie (mit Insolvenz) ist wesentlich steiler. Der Effekt der Schuldenquote ist in dieser Gruppe stärker.

6.4 Modellgüte und LUNG-Bedingungen

Wie verlässlich sind unsere gezogenen Schlüsse?

6.4.1 Das Bestimmtheitsmaß (R^2)

Das R^2 gibt an, welcher Anteil der Streuung (Varianz) in der Zielvariablen durch unser Modell erklärt wird.

```
glance(modell_interaktion) |> select(r.squared, adj.r.squared, p.value)
```

```
# A tibble: 1 x 3
  r.squared adj.r.squared p.value
  <dbl>      <dbl>      <dbl>
1    0.146        0.146        0
```

Unser Modell erklärt nur knapp 2,5 % ($R^2 \approx 0,025$) der Variabilität in den Zinssätzen. Die Effekte sind zwar hochsignifikant (p-Wert < 0.05), aber es gibt offensichtlich noch viele andere Variablen (wie den Credit Score), die den Zinssatz massiv beeinflussen.

*(Hinweis: Warum wir bei multiplen Modellen immer auf das **adj.r.squared** (Angepasstes R-Quadrat) schauen sollten, klären wir in Kapitel 5 zur Modellauswahl).*

6.4.2 Die LUNG-Bedingungen prüfen

Wir dürfen die p-Werte unseres Modells nur vertrauen, wenn vier technische Bedingungen (LUNG) erfüllt sind:

1. **L - Linearität:** Der Zusammenhang muss linear sein.
2. **U - Unabhängigkeit:** Die Beobachtungen müssen voneinander unabhängig sein.
3. **N - Normalverteilte Residuen:** Die Residuen (die Vorhersagefehler: $e_i = y_i - \hat{y}_i$) sollten annähernd glockenförmig verteilt sein.
4. **G - Gleiche Varianz (Homoskedastizität):** Die Streuung der Residuen sollte konstant bleiben, egal ob die Vorhersage klein oder groß ist.

```
resid_datan <- augment(modell_interaktion)

p1 <- ggplot(resid_datan, aes(x = .resid)) +
  geom_histogram(bins = 30, fill = "steelblue", color = "white") +
  theme_minimal() + labs(title = "Verteilung der Residuen")

p2 <- ggplot(resid_datan, aes(x = .fitted, y = .resid)) +
  geom_point(alpha = 0.3) + geom_hline(yintercept = 0, linetype = "dashed") +
  theme_minimal() + labs(title = "Residuen vs. Vorhersagewerte")

p1 + p2
```

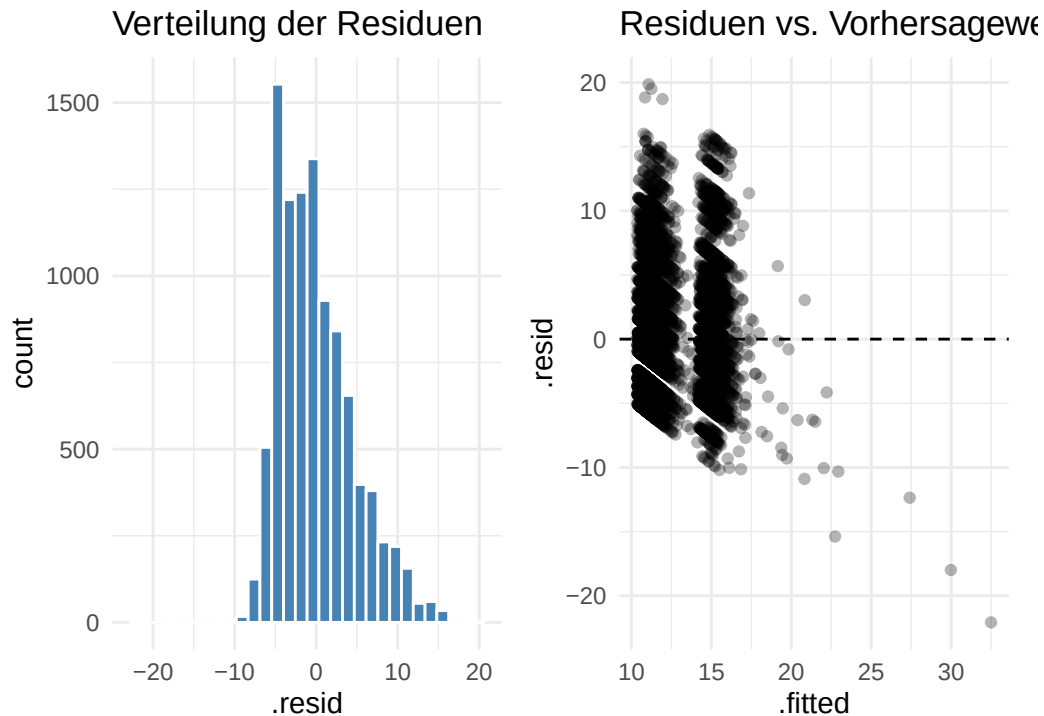



Abbildung 6.4: Links: Prüfung auf Normalität (N). Rechts: Prüfung auf Gleiche Varianz (G).

Diagnose: Die Residuen sind rechtssteil (nicht perfekt normal) und im Streudiagramm sehen wir diagonale Schnittkanten (oft ein Zeichen für harte Grenzen in den Originaldaten, z.B. wenn Zinssätze nicht beliebig klein werden können). Das Modell stößt hier an methodische Grenzen.

⚠ Multikollinearität (Achtung Falle!)

Wenn Sie in ein multiples Modell Prädiktoren aufnehmen, die stark *untereinander* korrelieren (z.B. “Größe des linken Beins” und “Größe des rechten Beins” zur Vorhersage der Körpergröße), gerät die Mathematik der OLS-Regression ins Straucheln. Dies nennt man **Multikollinearität**.

Koeffizienten wechseln plötzlich ihr Vorzeichen oder werden insignifikant. Das Modell als Ganzes bleibt oft stabil, aber die *Interpretation der einzelnen marginalen Effekte* wird extrem unzuverlässig, weil man in der Realität das linke Bein nicht verlängern kann, während man das rechte konstant hält.

7 Logistische Regression

8 Logistische Regression

i Kompetenzziele für dieses Kapitel

Nach der Bearbeitung dieses Kapitels können Sie:

1. **Grenzen der linearen Regression:** Erklären, warum OLS für binäre Zielvariablen (0/1) ungeeignet ist.
2. **Logistische Modellierung:** Modelle für binäre Daten in R (`glm(..., family = "binomial")`) anpassen und die Signifikanz der Prädiktoren ablesen.
3. **Nicht-lineare marginale Effekte:** Log-Odds in interpretierbare Wahrscheinlichkeiten umrechnen und verstehen, warum der marginale Effekt in der logistischen Regression nicht konstant ist.

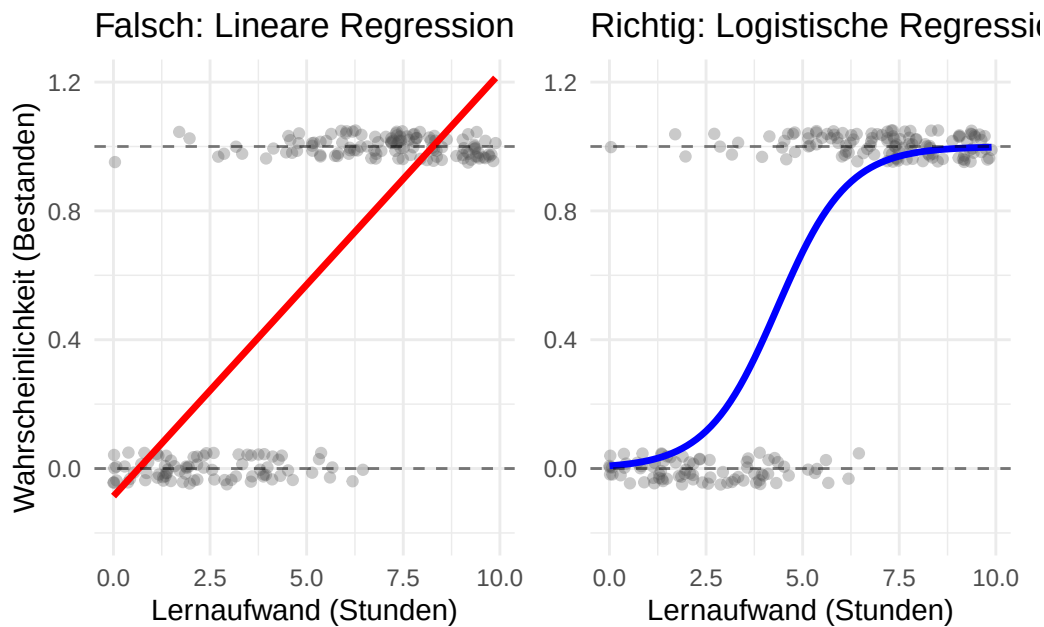
Oft stehen wir vor Forschungsfragen, bei denen das Ergebnis keine kontinuierliche Zahl (wie ein Preis oder eine Länge) ist, sondern eine **kategoriale Entscheidung mit zwei Ausprägungen (Ja/Nein)**:

- Klickt ein Nutzer auf die Anzeige? (Ja/Nein)
- Fällt ein Bauteil aus? (Ja/Nein)
- Erhält eine Bewerbung einen Rückruf? (Ja/Nein) Wir kodieren diese Ergebnisse als 1 (Erfolg/Ereignis tritt ein) und 0 (Misserfolg).

8.1 Warum die lineare Regression hier scheitert

Wenn wir versuchen, eine Zielvariable, die nur aus 0 und 1 besteht, mit der bekannten linearen Regression (`lm()`) zu modellieren, stoßen wir auf massive Probleme.

Visuelle Intuition: Die Gerade vs. die S-Kurve



Die rote Linie (OLS) schießt für extrem hohen oder niedrigen Lernaufwand über die Grenzen von 100 % und 0 % hinaus. Eine Wahrscheinlichkeit von 120 % oder -15 % ist mathematisch unsinnig. Die blaue Kurve (Logistische Regression) nähert sich asymptotisch an die Grenzen 0 und 1 an.

8.2 Das Logistische Modell

Um sicherzustellen, dass unsere Vorhersagen immer zwischen 0 und 1 bleiben, modellieren wir nicht direkt die Wahrscheinlichkeit p , sondern transformieren sie. Wir nutzen den **Logit** (den natürlichen Logarithmus der Chancen, auch *Log-Odds* genannt).

Unsere Regressionsgleichung lautet nun:

$$\log_e \left(\frac{p}{1-p} \right) = b_0 + b_1 x_1 + b_2 x_2 + \dots + b_k x_k$$

Das Geniale an dieser Transformation: Die linke Seite kann nun Werte von $-\infty$ bis $+\infty$ annehmen. Wir können die rechte Seite exakt so aufbauen wie bei der klassischen linearen Regression.

8.2.1 Operative Umsetzung in R (glm)

Da OLS hier nicht mehr funktioniert, nutzen wir die Funktion für **Generalized Linear Models** (`glm()`) und sagen R ausdrücklich, dass die Fehlerverteilung binomial ist.

Wir analysieren experimentelle Daten (`resume`). Forscher schickten gefälschte Lebensläufe an echte Stellenanzeigen. Die Lebensläufe unterschieden sich zufällig in Berufserfahrung, Auszeichnungen und dem Namen, der entweder typischerweise weiß oder schwarz klang. Zielvariable ist `erhaltener_rückruf` (1 = Ja, 0 = Nein).

```
# Modelliere: Rückruf anhand von Ethnie und Berufserfahrung
modell_logit <- glm(erhaltener_rückruf ~ ethnie + years_experience,
                   data = resume,
                   family = "binomial")

tidy(modell_logit) |> select(term, estimate, p.value)
```

```
# A tibble: 3 x 3
  term          estimate p.value
<chr>          <dbl>    <dbl>
1 (Intercept)    -3.00  2.37e-148
2 ethnieWeiß      0.438  4.56e- 5
3 years_experience 0.0391 2.14e- 5
```

Wie Sie diese Tabelle lesen:

1. **Inferenz (p.value):** Wir lesen dies exakt wie bei der linearen Regression. Die Koeffizienten für `ethnieWeiß` und `years_experience` sind beide signifikant ($p < 0.05$). Beide Prädiktoren haben einen statistisch nachweisbaren Effekt auf die Rückrufquote.
2. **Deskription (estimate): Vorsicht!** Diese Zahlen (z.B. 0,438 für die Ethnie) sind **keine** Wahrscheinlichkeiten. Es sind die Änderungen der *Log-Odds*. Ein positiver Wert bedeutet: Die Wahrscheinlichkeit steigt. Ein negativer Wert bedeutet: Sie sinkt.

Ein weißer Name erhöht also die Wahrscheinlichkeit eines Rückrufs (*ceteris paribus*). Aber um wie viel Prozent genau? Das können wir an dieser Zahl nicht direkt ablesen.

8.3 Marginale Effekte in der logistischen Regression

In Kapitel 3 haben wir gelernt, dass der marginale Effekt die partielle Ableitung der Vorhersage nach der Variable x ist ($\frac{\partial \hat{y}}{\partial x}$).

Bei der klassischen OLS-Regression war das eine Konstante (die Linie ist überall gleich steil). Bei der logistischen Regression ist unsere Vorhersagekurve jedoch ein “S”.

Visuelle Intuition: Der nicht-konstante marginale Effekt

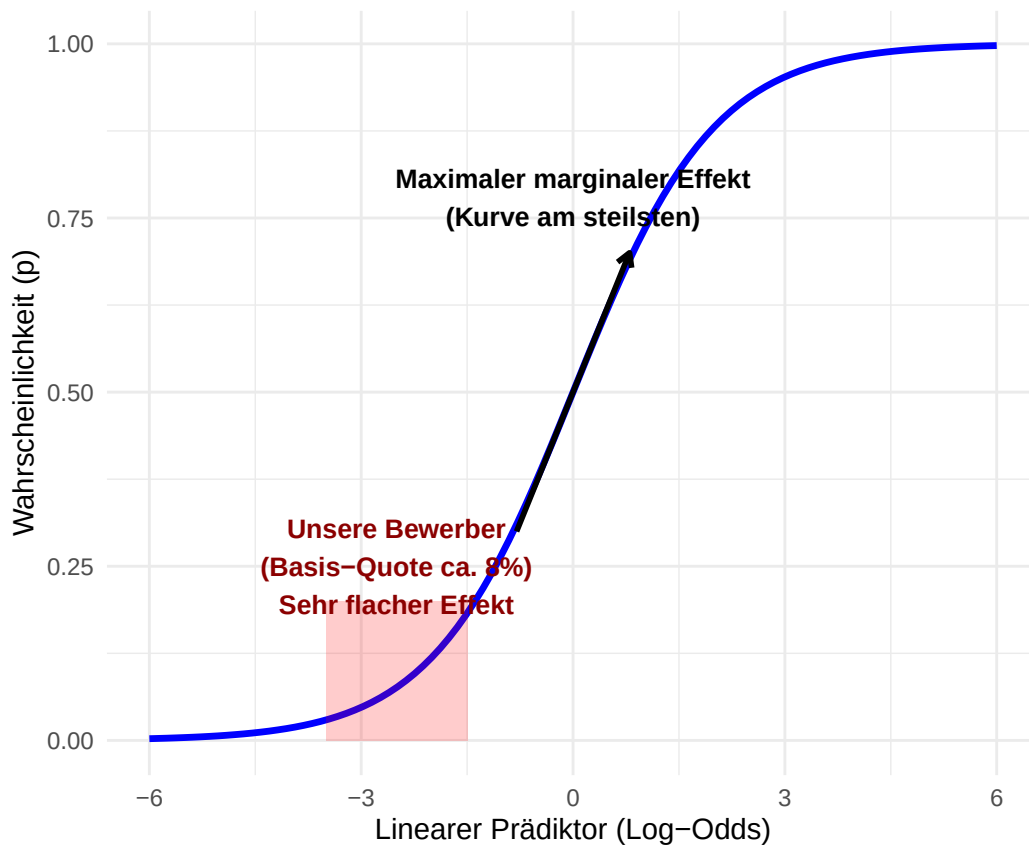


Abbildung 8.1: Die logistische S-Kurve. Die Steigung (der marginale Effekt) hängt stark davon ab, wo man sich auf der Kurve befindet.

- **In der Mitte der S-Kurve** (Wahrscheinlichkeit bei 50 %) ist die Kurve am steilsten. Hier hat eine Änderung der Prädiktoren den maximalen marginalen Effekt.
- **In den extrem flachen Enden** (z.B. nahe 0 % oder 100 %) hat eine Änderung der Prädiktoren kaum noch einen Effekt (die Ableitung ist fast 0).

- **Unsere Fallstudie:** Rückrufe auf Bewerbungen sind generell selten. Unsere Bewerber (rotes Feld) stecken im unteren, flachen Tail der S-Kurve fest.

Fazit: In der logistischen Regression ist der marginale Effekt **niemals konstant**. Er hängt zwingend davon ab, wie wahrscheinlich das Ereignis *vor* der Änderung von x bereits war.

8.3.1 Wahrscheinlichkeiten berechnen

Um aus den abstrakten Log-Odds greifbare Wahrscheinlichkeiten (p) zu machen, müssen wir die Modellgleichung umstellen:

$$p = \frac{e^{\text{Modellgleichung}}}{1 + e^{\text{Modellgleichung}}}$$

In R müssen wir das nicht per Hand rechnen. Wir nutzen die `predict()` Funktion mit dem Argument `type = "response"`, um Wahrscheinlichkeiten zu generieren.

8.3.2 Fallstudie: Die Auswirkung von Diskriminierung

Bauen wir ein voll spezifiziertes multiples Modell, um zu messen, wie stark sich ein “weiß klingender” Name gegenüber einem “schwarz klingenden” Namen auswirkt, wenn alle anderen Merkmale (Erfahrung, Abschluss, Auszeichnungen) im Lebenslauf identisch sind.

```
modell_diskriminierung <- glm(erhaltener_rückruf ~ ethnie + geschlecht + years_experience + a
                                data = resume,
                                family = "binomial")

tidy(modell_diskriminierung) |> select(term, estimate, p.value)
```

```
# A tibble: 5 x 3
  term          estimate p.value
<chr>          <dbl>    <dbl>
1 (Intercept)   -2.98  5.17e-138
2 ethnieWeiß    0.440  4.49e- 5
3 geschlechtMann -0.100  4.39e- 1
4 years_experience 0.0331 3.90e- 4
5 auszeichnungen1 0.756  3.22e- 5
```

Der Koeffizient für `ethnieWeiß` ist 0,442 (p-Wert hochsignifikant). Die Diskriminierung ist statistisch nachweisbar. Um den Effekt greifbar zu machen, lassen wir R die Wahrscheinlichkeit für zwei identische fiktive Bewerbungen vorhersagen:

- Bewerbung A: Schwarz, Mann, 10 Jahre Erfahrung, keine Auszeichnungen.
- Bewerbung B: Weiß, Mann, 10 Jahre Erfahrung, keine Auszeichnungen.

```
# 1. Wir bauen uns einen fiktiven Datensatz für die zwei Bewerber
neue_bewerber <- data.frame(
  ethnie = c("Schwarz", "Weiß"),
  geschlecht = c("Mann", "Mann"),
  years_experience = c(10, 10),
  auszeichnungen = as.factor(c(0, 0)) # Muss als Faktor übergeben werden
)

# 2. Wir lassen das Modell die Wahrscheinlichkeiten (response) vorhersagen
predict(modell_diskriminierung, newdata = neue_bewerber, type = "response")
```

```
      1      2
0.05995200 0.09008557
```

Interpretation des marginalen Effekts: * Der Bewerber mit dem schwarz klingenden Namen (Bewerbung A) hat eine Rückrufwahrscheinlichkeit von ca. **6,4 %**. * Der Bewerber mit exakt demselben Lebenslauf, aber einem weiß klingenden Namen (Bewerbung B) hat eine Rückrufwahrscheinlichkeit von ca. **9,6 %**.

Das bedeutet: Der marginale Effekt der Variable "Ethnie" bei genau diesem Bewerberprofil beträgt +3,2 Prozentpunkte. Relativ gesehen muss ein schwarzer Bewerber fast 50 % mehr Bewerbungen schreiben (da 9,6 % etwa das 1,5-fache von 6,4 % ist), um die gleiche Anzahl an Rückrufen zu erhalten.

8.4 Zusammenfassung der Entscheidungsstrukturen

1. **Modellauswahl:** Ist die Zielvariable binär (0/1), wechseln Sie von `lm()` zu `glm(..., family = "binomial")`.
2. **Signifikanz prüfen:** Lesen Sie den p-Wert in der `tidy()` Tabelle exakt wie bei der linearen Regression, um zu prüfen, ob ein Prädiktor das Modell signifikant verbessert.
3. **Effekte greifbar machen:** Interpretieren Sie die Koeffizienten (`estimate`) nicht direkt als Wahrscheinlichkeiten. Nutzen Sie `predict(..., type = "response")`, um Profile zu erstellen und die tatsächlichen, nicht-konstanten marginalen Effekte auf die Wahrscheinlichkeit zu quantifizieren.

9 Modellselektion

10 Modellselektion

i Kompetenzziele für dieses Kapitel

Nach der Bearbeitung dieses Kapitels können Sie:

1. **Die Overfitting-Falle erkennen:** Verstehen, warum das Hinzufügen von immer mehr Variablen die Vorhersagekraft eines Modells auf neuen Daten zerstören kann.
2. **Metriken zur Modellauswahl nutzen:** Das *Angepasste R^2* (für OLS) und das *AIC* (für logistische Regression) anwenden, um sparsame Modelle zu identifizieren.
3. **Kreuzvalidierung (Cross-Validation) anwenden:** Den k -fachen Kreuzvalidierungs-Workflow in R programmieren, um die echte, out-of-sample Vorhersagegüte eines Modells zu messen.

10.1 Das Problem: Overfitting (Überanpassung)

Ein weit verbreiteter Irrtum in der Datenanalyse ist: *“Je mehr Variablen ich in mein Modell werfe, desto genauer wird meine Vorhersage.”*

Mathematisch gesehen steigt das reguläre Bestimmtheitsmaß (R^2) immer an (oder bleibt im schlechtesten Fall gleich), wenn wir einen neuen Prädiktor hinzufügen – selbst wenn dieser Prädiktor absoluter Quatsch ist. Das Modell beginnt, das statistische Rauschen der spezifischen Stichprobe **auswendig zu lernen**, anstatt den echten zugrundeliegenden Trend zu **verstehen**. Diesen Effekt nennt man **Overfitting**.

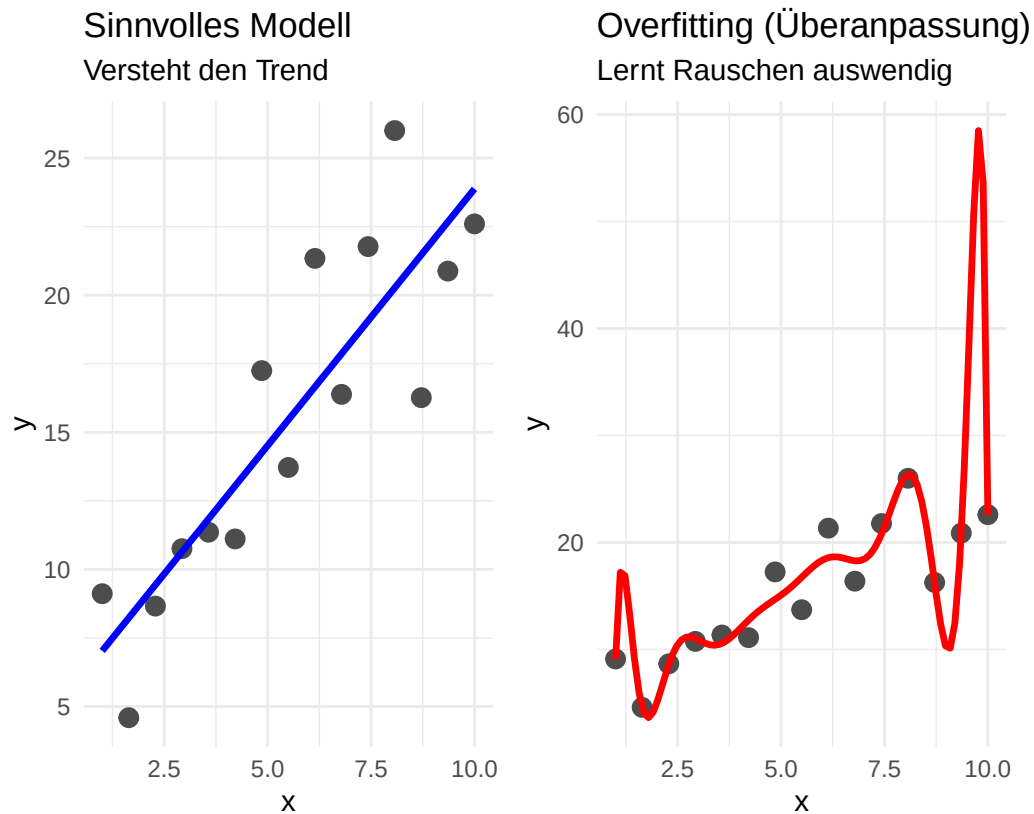


Abbildung 10.1: Links: Ein einfaches Modell, das den generellen Trend versteht. Rechts: Ein überangepasstes Modell (Overfitting), das die Datenpunkte auswendig lernt, aber bei neuen Daten massiv scheitern wird.

Das rote Modell berührt fast jeden Punkt (hohes R^2). Aber was passiert, wenn wir einen neuen Datenpunkt bei $x = 8$ vorhersagen wollen? Das Modell würde völlig absurde Werte liefern. **Unser Ziel ist ein “sparsames” Modell:** So komplex wie nötig, aber so einfach wie möglich.

10.2 Kennzahlen für die Modellauswahl

Um Overfitting zu verhindern, brauchen wir Metriken, die das Modell “bestrafen”, wenn unnötige Variablen hinzugefügt werden.

10.2.1 1. Das Angepasste R^2 (Für Lineare Regression)

Während das normale R^2 bei jedem neuen Prädiktor wächst, korrigiert das **Angepasste R^2** (R^2_{adj}) diesen Wert nach unten, basierend auf der Anzahl der Variablen im Modell (k).

- Fügen wir eine Variable hinzu, die das Modell stark verbessert, steigt das R^2_{adj} .
- Fügen wir eine nutzlose Variable hinzu, greift die mathematische Strafe, und das R^2_{adj} **sinkt**.

Bei der **Rückwärtsselektion (Backward Selection)** starten wir mit dem vollen Modell (alle Variablen) und entfernen schrittweise die Variablen, die das R^2_{adj} nicht verbessern oder sogar verschlechtern.

10.2.2 2. Das AIC (Für Logistische Regression)

Bei der logistischen Regression (siehe Kapitel 4) existiert kein klassisches R^2 . Stattdessen nutzen wir das **Akaike-Informationskriterium (AIC)**.

- Auch das AIC bestraft Komplexität (Anzahl der Variablen).
- **Regel:** Im Gegensatz zum R^2 (wo höher besser ist), gilt beim AIC: **Je kleiner der AIC-Wert, desto besser das Modell.**

```
# Beispiel für den AIC-Vergleich bei der Spam-Erkennung
modell_spam_klein <- glm(spam ~ to_multiple, data = email, family = "binomial")
modell_spam_gross <- glm(spam ~ to_multiple + cc + urgent_subj, data = email, family = "binomial")

# glance() gibt uns die Metriken aus
glance(modell_spam_klein) |> select(AIC, nobs)
```

```
# A tibble: 1 x 2
  AIC  nobs
<dbl> <int>
1 2376.  3921
```

```
glance(modell_spam_gross) |> select(AIC, nobs)
```

```
# A tibble: 1 x 2
  AIC  nobs
<dbl> <int>
1 2368.  3921
```

Das größere Modell hat den kleineren AIC-Wert. Es ist trotz der Strafe für mehr Variablen das bessere Modell.

10.3 Die ultimative Prüfung: Kreuzvalidierung (Cross-Validation)

Das Angepasste R^2 und das AIC sind gute theoretische Hilfsmittel. Die harte Realität in der Data Science ist jedoch eine andere: **Wie gut ist mein Modell bei Daten, die es noch nie zuvor gesehen hat?**

Die Lösung heißt **Kreuzvalidierung (CV)**. Anstatt das Modell auf allen Daten zu trainieren und zu testen (was Schummeln wäre), teilen wir die Daten auf:

1. **Trainingsdaten:** Ein Teil der Daten, auf dem wir die Modellparameter $(b_0, b_1, \dots)$ schätzen.
2. **Testdaten (Holdout):** Ein anderer Teil, an dem wir prüfen, wie groß unsere Vorhersagefehler ($e = y - \hat{y}$) tatsächlich sind.

10.3.1 Workflow: k -fache Kreuzvalidierung

Bei der k -fachen CV (z.B. $k = 4$) teilen wir die Daten in 4 gleich große Blöcke ("Folds"). Wir trainieren das Modell auf 3 Blöcken und testen es am 4. Block. Das wiederholen wir 4-mal, sodass jeder Block genau einmal der Test-Block war.

10.3.2 Praxis-Beispiel: Pinguin-Körpermasse

Wir wollen die Masse von Pinguinen (`body_mass_g`) vorhersagen. * **Modell A (Klein):** Nutzt nur die Schnabellänge (`bill_length_mm`). * **Modell B (Groß):** Nutzt Schnabellänge, Schnabeltiefe, Flossenlänge, Geschlecht und Spezies.

Welches Modell liefert in der Realität die besseren Vorhersagen? Wir programmieren einen 4-fachen CV-Workflow für Modell B:

```

set.seed(47)

# 1. Daten in 4 Folds aufteilen (erzeugt eine Liste mit Zeilen-Indizes für jeden Fold)
folds <- createFolds(penguins$body_mass_g, k = 4)

# 2. Leeren Container für unsere Ergebnisse vorbereiten
cv_ergebnisse <- data.frame()

# 3. Der CV-Loop: Für jeden der 4 Folds einmal durchlaufen
for(i in 1:4) {
  test_indizes <- folds[[i]]

  # Daten aufteilen
  test_daten <- penguins[test_indizes, ]
  train_daten <- penguins[-test_indizes, ] # Minus bedeutet: Alles AUßER test_indizes

  # Modell NUR auf den Trainingsdaten anpassen
  modell_cv <- lm(body_mass_g ~ bill_length_mm + bill_depth_mm + flipper_length_mm + sex + species,
                  data = train_daten)

  # Vorhersage für die unabhängigen Testdaten machen
  vorhersage <- predict(modell_cv, newdata = test_daten)

  # Ergebnisse speichern
  fold_ergebnis <- data.frame(
    Fold = paste("Fold", i),
    Beobachtet = test_daten$body_mass_g,
    Vorhergesagt = vorhersage,
    Fehler = test_daten$body_mass_g - vorhersage
  )
  cv_ergebnisse <- bind_rows(cv_ergebnisse, fold_ergebnis)
}

# 4. Den Gesamtfehler berechnen (Kreuzvalidierte Summe der Fehlerquadrate - CV SSE)
cv_sse_gross <- sum(cv_ergebnisse$Fehler^2)

```

Würden wir diesen Code für **Modell A** (nur Schnabellänge) ausführen, erhielten wir einen Gesamtfehler (CV SSE) von ca. **141.552.000**. Der Gesamtfehler für unser **Modell B** (aus dem Code oben) beträgt hingegen nur ca. **27.728.000**.

Der Vorhersagefehler ist massiv geschrumpft! Das größere Modell ist hier eindeutig überlegen und leidet nicht unter Overfitting.

Visuelle Intuition: Die Vorhersagefehler der Kreuzvalidierung

Wir können uns die Fehler ($\text{Fehler} = \text{Beobachtet} - \text{Vorhergesagt}$) für unsere 4 Test-Folds aus Modell B grafisch ansehen.

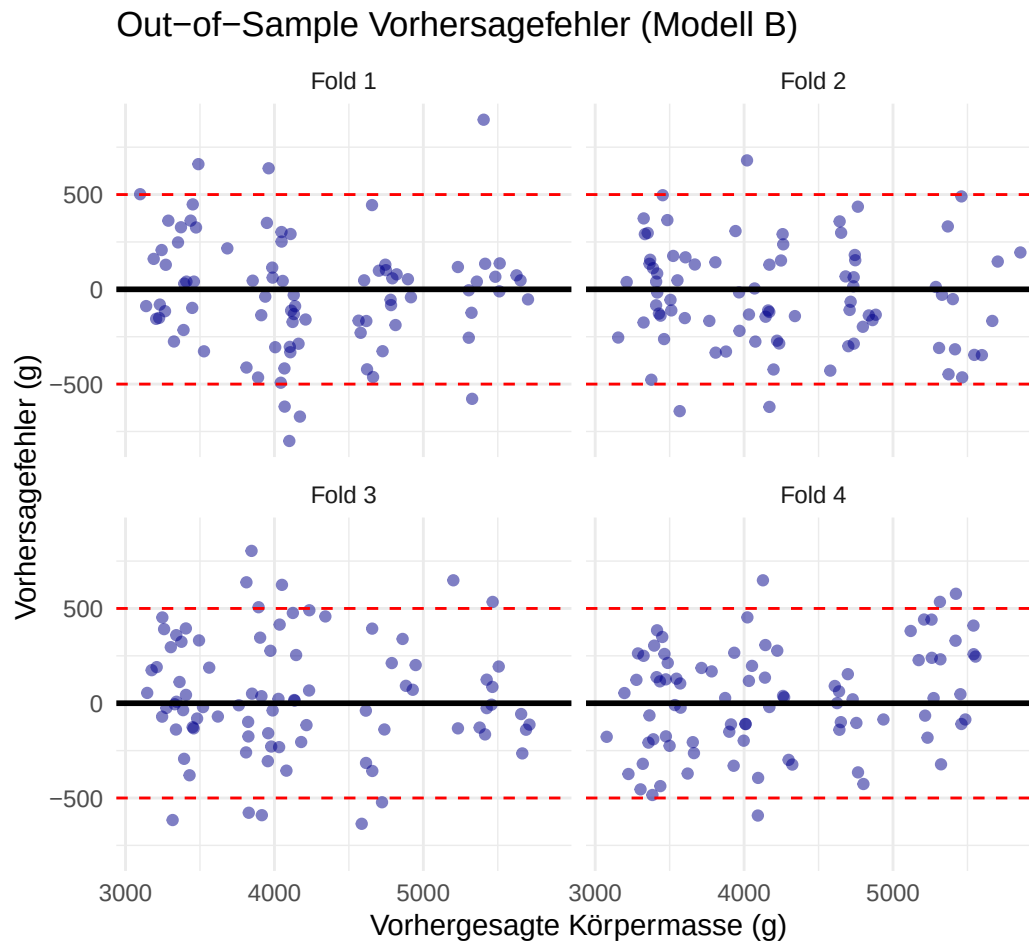


Abbildung 10.2: Die Vorhersagefehler in den 4 Test-Durchläufen (Folds) von Modell B. Nahezu alle Vorhersagen liegen innerhalb eines Fensters von ± 500 Gramm.

Die Grafik zeigt, dass das Modell extrem stabil ist: Egal welcher Fold gerade der Test-Datensatz war, die Fehler streuen verlässlich gleichmäßig um die Nulllinie. Hätten wir Modell A geplottet, würden die Punkte weit über die roten $\pm 500g$ -Linien hinaus in den Bereich von über $\pm 1000g$ streuen.

Der Brückenschlag: Sie betreiben bereits Machine Learning!

Vielleicht ist Ihnen das Wort „Machine Learning“ (Maschinelles Lernen) in den Medien oder anderen Kursen schon begegnet. Die gute Nachricht ist: Mit dem Verständnis der Kreuzvalidierung haben Sie gerade das fundamentale Kernprinzip des modernen Machine Learnings (ML) gemeistert.

Im Kern macht ein hochentwickelter ML-Algorithmus bei Spotify, Netflix oder in der Kreditrisikoprüfung nichts anderes als das, was Sie in diesem Kapitel programmiert haben:

1. **Training Phase:** Der Algorithmus sucht in historischen Daten nach Mustern (exakt wie R die Koeffizienten b_i im OLS-Modell schätzt).
2. **Testing Phase:** Das gelernte Modell muss sich an völlig neuen, isolierten Daten beweisen (Out-of-Sample), um zu zeigen, ob es echte Zusammenhänge verstanden oder nur Rauschen auswendig gelernt hat (Overfitting).

Ein lineares oder logistisches Regressionsmodell, das mittels Kreuzvalidierung auf seine Vorhersagekraft geprüft wird, ist die solide Basis fast jeder professionellen Machine-Learning-Pipeline. Sie haben soeben den Schritt von der reinen Datenbeschreibung (Deskription) zur echten prädiktiven Analytik (Predictive Analytics) vollzogen.

10.4 Fazit: Die Strategie eines Data Scientists

Modellbildung ist keine Einbahnstraße. Ein professioneller Workflow sieht so aus:

1. **Fundament:** Starten Sie mit der Fachlogik (Welche Variablen ergeben theoretisch Sinn?).
2. **Filterung:** Werfen Sie Variablen raus, die das Angepasste R^2 verschlechtern oder den AIC in die Höhe treiben.
3. **Realitätscheck:** Nutzen Sie Kreuzvalidierung, um die endgültige Vorhersagekraft (bei numerischen Zielen den $CV\ SSE$, bei logistischen Zielen die *Accuracy*) zu messen und Ihr Modell vor der Praxis zu schützen.

11 Fragebogendaten und Latente Konstrukte

12 Fragebogendaten und Latente Konstrukte

i Kompetenzziele für dieses Kapitel

Nach der Bearbeitung dieses Kapitels können Sie:

1. **Supervised vs. Unsupervised Learning:** Den Unterschied zwischen prädiktiver Modellierung (Regression) und strukturentdeckenden Verfahren erkennen.
2. **Latente Konstrukte verstehen:** Das Konzept hinter unbeobachtbaren Variablen (z. B. “Zufriedenheit”, “Stress”) und deren Messung durch manifeste Fragebogen-Items erklären.
3. **Dimensionsreduktion anwenden:** Eine Explorative Faktoranalyse (EFA) in R durchführen, Scree-Plots interpretieren und Faktorwerte (Scores) berechnen, um diese als saubere Prädiktoren in Regressionsmodellen zu nutzen.

In den Kapiteln 3 bis 5 haben wir uns intensiv mit der **Regression** beschäftigt. Wir hatten immer eine klare, bekannte Zielvariable y (z. B. Körpermasse, Zinssatz, Rückruf-Status), die wir durch Prädiktoren x vorhersagen wollten.

In der modernen Data Science nennt man diesen Ansatz **Supervised Learning (Überwachtes Lernen)**: Das Modell wird anhand von konkreten “Lösungen” (den beobachteten y -Werten) trainiert und korrigiert seine Fehler.

12.1 Der Paradigmenwechsel: Unsupervised Learning

Oft stehen wir in der empirischen Forschung (insbesondere bei Umfragen und Fragebögen) vor einem völlig anderen Problem: Wir haben gar kein konkretes y , das wir vorhersagen wollen. Stattdessen haben wir einen riesigen Datensatz an x -Variablen und wollen wissen: **Gibt es hier verborgene Muster oder Strukturen?**

Dieser Ansatz heißt **Unsupervised Learning (Unüberwachtes Lernen)**. Wir geben dem Algorithmus keine Lösungen vor, sondern lassen ihn die Daten selbstständig ordnen. Eines der wichtigsten Verfahren dafür ist die **Dimensionsreduktion** – und im Kontext von Fragebögen speziell die **Faktoranalyse**.

12.1.1 Das Praxis-Problem: Multikollinearität im Fragebogen

Stellen Sie sich vor, Sie werten eine Mitarbeiterumfrage aus. Sie haben 15 Fragen auf einer 5-stufigen Likert-Skala gestellt, die alle grob in Richtung “Arbeitszufriedenheit” gehen (z.B. “Ich mag mein Team”, “Mein Chef kommuniziert gut”, “Die Kaffeemaschine funktioniert”).

Wenn Sie nun versuchen, die Produktivität (y) der Mitarbeiter durch diese 15 Fragen (x_1 bis x_{15}) in einem multiplen Regressionsmodell (`lm`) vorherzusagen, tappen Sie in eine massive Falle: **Multikollinearität** (siehe Kapitel 3). Wer bei “Ich mag mein Team” eine 5 ankreuzt, kreuzt oft auch bei “Gutes Arbeitsklima” eine 5 an. Die Variablen korrelieren so stark untereinander, dass die OLS-Mathematik zusammenbricht und die marginalen Effekte der einzelnen Fragen uninterpretierbar werden.

12.1.2 Die Lösung: Latente Konstrukte

Anstatt 15 hochkorrelierte Fragen ins Modell zu zwingen, drehen wir die Logik um: Wir gehen davon aus, dass es ein unsichtbares, **latentes Konstrukt** (die wahre, innere “Arbeitszufriedenheit”) gibt. Dieses Konstrukt können wir nicht direkt messen. Wir können nur die 15 beobachtbaren Reaktionen (die **manifesten Variablen**) messen, die durch dieses Konstrukt verursacht werden.

Die **Explorative Faktoranalyse (EFA)** sucht nach genau diesen verborgenen latenten Variablen (den “Faktoren”). Sie fasst hochkorrelierte Einzelfragen zu wenigen, unkorrelierten Über-Dimensionen zusammen. Aus 15 Fragen wird so vielleicht ein einziger “Zufriedenheits-Score”, den wir danach sauber und gefahrlos in unser Regressionsmodell einbauen können.

12.2 Durchführung einer Faktoranalyse in R

Wir nutzen den `bfi`-Datensatz aus dem `psych`-Paket. Er enthält die Antworten von über 2.000 Personen auf 25 Fragen zu ihrer Persönlichkeit. Wir wollen prüfen, ob sich diese 25 Einzelfragen auf zugrundeliegende Persönlichkeits-Faktoren (Dimensionen) reduzieren lassen.

12.2.1 Schritt 1: Wie viele Faktoren gibt es? (Der Scree-Plot)

Bevor wir Faktoren extrahieren, müssen wir entscheiden, *wie viele* Faktoren mathematisch sinnvoll sind. Jede Frage ist eine Dimension. Wir haben 25 Dimensionen. Ein Faktor ist nur dann nützlich, wenn er wesentlich mehr Information (Varianz) bündelt als eine einzelne, isolierte Frage.

Das Standard-Werkzeug dafür ist der **Scree-Plot** in Kombination mit der **Parallelanalyse**.

```
fa.parallel(fragebogen_daten, fa = "fa", fm = "minres", main = "Scree-Plot und Parallelanalyse")
```

Scree-Plot und Parallelanalyse

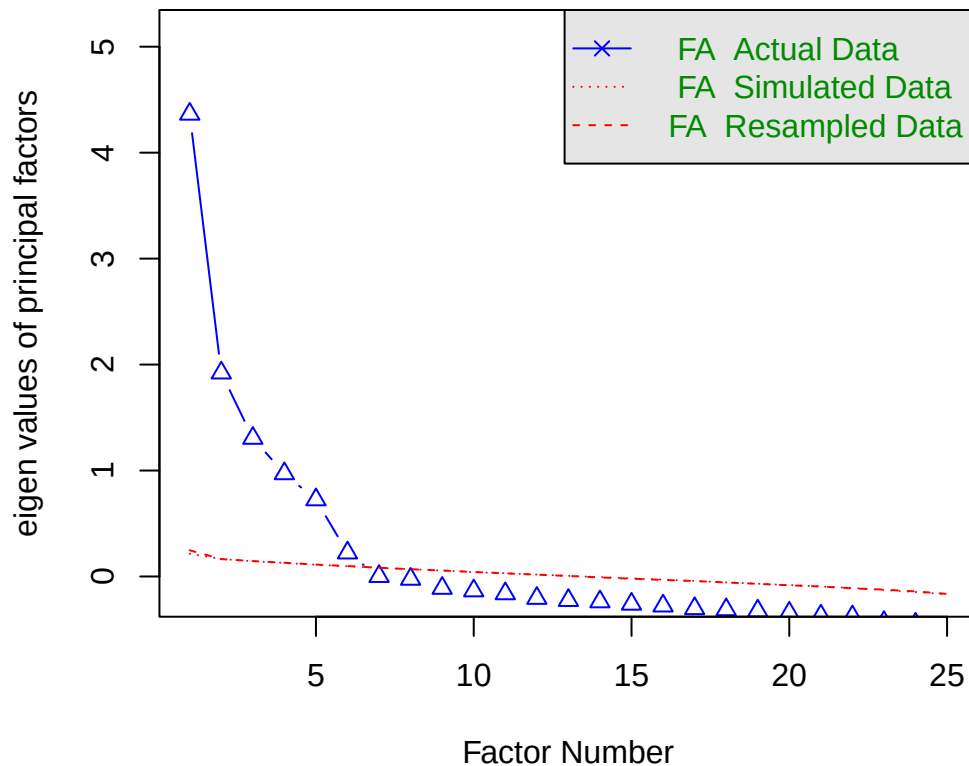


Abbildung 12.1: Der Scree-Plot mit Parallelanalyse. Wir suchen den Punkt, an dem die blauen Dreiecke (echte Daten) unter die rote Linie (Zufallsdaten) fallen.

Parallel analysis suggests that the number of factors = 6 and the number of components = 1

Wie Sie diesen Plot lesen:

- Die Y-Achse (Eigenwerte) zeigt, wie viel “Informationskraft” ein Faktor hat. Ein Wert von über 1 bedeutet: Dieser Faktor bündelt mehr Information als eine einzelne Frage.
- Die **blaue Linie** zeigt unsere tatsächlichen Faktoren.

- Die **rote, gestrichelte Linie** zeigt, wie die Faktoren aussehen würden, wenn unsere Daten reiner Zufall (ohne jedes Muster) wären.
- **Die Entscheidungsregel:** Wir behalten nur die Faktoren, deren blaue Dreiecke *über* der roten Zufallslinie (und idealerweise über 1.0) liegen.

Das Diagramm ist eindeutig: Nach dem 5. Faktor flacht die Kurve massiv ab (der “Geröll”-Knick, englisch *scree*). Es gibt **5 latente Konstrukte** in diesen 25 Fragen. (Psychologen erkennen hier sofort die “Big Five” der Persönlichkeitsforschung wieder).

12.2.2 Schritt 2: Die Faktoren extrahieren (Faktorladungen)

Nachdem wir wissen, dass wir 5 Faktoren suchen, lassen wir R berechnen, welche der 25 Fragen zu welchem dieser 5 Faktoren gehören.

(Hinweis: Wir nutzen hier eine “oblique” Rotation, `rotate = "oblimin"`. Das erlaubt den Faktoren, untereinander leicht zu korrelieren – was in der psychologischen und wirtschaftlichen Realität fast immer der Fall ist).

```
# Führe die Faktoranalyse mit 5 Faktoren durch
modell_efa <- fa(fragebogen_daten, nfactors = 5, rotate = "oblimin", fm = "minres")

# Zeige die Zuordnung (Ladungen) an. Zur besseren Lesbarkeit blenden wir
# schwache Ladungen (unter 0.3) aus.
print(modell_efa$loadings, cutoff = 0.3, sort = TRUE)
```

Loadings:

	MR2	MR5	MR4	MR1	MR3
N1	0.832				
N2	0.780				
N3	0.703				
E1		0.555			
E2		0.667			
E4		-0.587			
C1			0.555		
C2			0.669		
C3			0.574		
C4			-0.644		
C5			-0.563		
A2				0.656	
A3				0.678	
A5				0.537	

O1				0.518	
O3				0.619	
O5				-0.540	
A1			-0.435		
A4			0.448		
E3		-0.408		0.302	
E5		-0.418			
N4	0.471	0.405			
N5	0.483				
O2				-0.473	
O4		0.335		0.363	

	MR2	MR5	MR4	MR1	MR3
SS loadings	2.504	1.962	1.979	1.890	1.562
Proportion Var	0.100	0.078	0.079	0.076	0.062
Cumulative Var	0.100	0.179	0.258	0.333	0.396

Wie Sie diese Matrix lesen:

- In den Spalten sehen Sie unsere 5 verborgenen Faktoren (MR2, MR1, MR3, MR5, MR4).
- In den Zeilen stehen unsere 25 Fragen (z.B. N1 bis N5).
- Die Zahlen sind die **Faktorladungen** (grob vergleichbar mit Korrelationskoeffizienten). Sie reichen von -1 bis +1.
- *Interpretation:* Die Fragen N1 bis N5 laden alle extrem hoch auf dem Faktor **MR2**. Sie messen also offensichtlich alle dasselbe latente Konstrukt (in der Psychologie: “Neurotizismus”). Die Fragen E1 bis E5 laden gemeinsam auf MR1 (“Extraversion”).

Wir haben das Chaos von 25 Einzelvariablen erfolgreich auf 5 klare, inhaltlich interpretierbare Dimensionen reduziert!

12.2.3 Schritt 3: Faktor-Scores für die Regression berechnen

Das ist der entscheidende Moment, der Unsupervised Learning wieder mit Supervised Learning (Kapitel 3) verbindet.

Anstatt nun 25 wackelige Fragen in unser nächstes Regressionsmodell zu werfen, können wir R anweisen, für jeden Probanden aus den Fragen seine 5 neuen **Faktor-Scores** zu berechnen.

```
# Extrahieren der berechneten Scores für jede Person
faktoren_scores <- as.data.frame(modell_efa$scores)

# Wir schauen uns die ersten 5 Probanden an:
head(faktoren_scores)
```

	MR2	MR5	MR4	MR1	MR3
61617	-0.2224204	-0.12783706	-1.32676882	-0.85525792	-1.6092012
61618	0.1617984	-0.46568423	-0.57221671	-0.07242355	-0.1722206
61620	0.6190712	-0.14137060	-0.04294803	-0.55241382	0.2334113
61621	-0.1169206	-0.05806742	-1.06320140	-0.09063267	-1.0615175
61622	-0.1737256	-0.46001004	-0.09947329	-0.71181405	-0.6608554
61623	0.1934194	-1.09198355	1.39949994	0.38874811	0.6242741

Was haben wir erreicht? Wir haben jetzt einen stark komprimierten, bereinigten Datensatz. Wenn wir nun z.B. untersuchen wollen, wie sich die Persönlichkeit auf das Gehalt (y) auswirkt, nutzen wir nicht mehr die 25 unübersichtlichen und kollinearen Einzelfragen. Wir nehmen einfach diese 5 neuen Variablen (MR1 bis MR5) und stecken sie in eine klassische multiple OLS-Regression (`lm(gehalt ~ MR1 + MR2 + ...)`).

12.3 Zusammenfassung: Die Daten-Pipeline

Die Faktoranalyse ist kein Selbstzweck, sondern oft der erste, entscheidende Schritt einer professionellen Daten-Pipeline bei Befragungen:

1. **Unsupervised Step (EFA):** Reduzieren Sie viele, unübersichtliche und hochkorrelierte Items aus dem Fragebogen auf wenige, aussagekräftige Faktoren (latente Konstrukte).
2. **Scoring:** Berechnen Sie für jede Beobachtung die Werte auf diesen neuen Faktoren.
3. **Supervised Step (OLS/Logit):** Nutzen Sie diese sauberen Faktor-Scores als Prädiktoren in Ihren Regressionsmodellen, um Zielvariablen vorhersagen, ohne in die Multikollinearitäts-Falle zu tappen.

13 Anhang: Quarto Survival Guide

Anhang: Quarto Survival Guide

Gerade bei intensiven Projektphasen – wie einer Data Challenge, für die Sie exakt zwei Wochen Zeit haben – darf die Software Sie nicht aufhalten. Sie brauchen einen Workflow, der Ihre statistischen Analysen direkt und reproduzierbar in professionelle Berichte oder Präsentationen übersetzt. Copy-and-Paste von R nach Word ist fehleranfällig und kostet wertvolle Zeit.

Dieser Guide ist kein umfassendes Handbuch. Er fokussiert sich auf die 20 % der Quarto-Funktionen, mit denen Sie 80 % Ihrer Abgaben und Präsentationen problemlos bewältigen.

13.1 Das Skelett (Der YAML-Header)

Jedes Quarto-Dokument beginnt mit einem sogenannten YAML-Header. Dieser Block steht ganz oben in der Datei, wird von --- umschlossen und steuert das übergeordnete Format. Hier entscheiden Sie mit einem einzigen Wort, ob RStudio einen Textbericht oder eine Präsentation generieren soll.

Für einen klassischen HTML-Bericht (Hausarbeit/Analyse):

```
---  
title: "Analyse der Kreditrisiken"  
author: "Ihr Name"  
format: html  
theme: spacelab  
toc: true  
---
```

(Tipp: `toc: true` generiert automatisch ein Inhaltsverzeichnis anhand Ihrer Überschriften).

Für eine interaktive Präsentation (Folien):

```
---
title: "Analyse der Kreditrisiken"
author: "Ihr Name"
format: revealjs
theme: simple
---
```

(Tipp: Wenn Sie *revealjs* nutzen, erzeugt jede Hauptüberschrift **##** automatisch eine neue Folie).

13.2 Die Markdown-Basics (Textformatierung)

Quarto nutzt Markdown für die Textformatierung. Sie benötigen keine Buttons für das Layout, sondern tippen einfache Sonderzeichen direkt in den Text. Das sind die absoluten Grundlagen, die Sie benötigen:

- **Überschriften:** Nutzen Sie Rauten (#).
 - # Ebene 1 (Hauptkapitel)
 - ## Ebene 2 (Unterkapitel oder neue Folie)
 - ### Ebene 3 (Abschnitt)
- **Hervorhebungen:** Nutzen Sie Sternchen.
 - ****Fettgedruckt**** wird zu **Fettgedruckt**
 - **Kursiv** wird zu *Kursiv*
- **Listen:** Nutzen Sie einfache Spiegelstriche oder Sternchen, gefolgt von einem Leerzeichen. Um eine Liste zu starten, muss davor zwingend eine Leerzeile stehen!
 - Punkt 1
 - Punkt 2
- **Links:** [Sichtbarer Text] (<https://www.link.de>)

13.3 Chunk-Kontrolle (Die Magie)

Die wahre Stärke von Quarto liegt in den “Code-Chunks”. Das sind isolierte Blöcke, in denen Ihr R-Code ausgeführt wird. Sie steuern über sogenannte **Hash-Pipe-Optionen** (`#|`), was der Leser am Ende im fertigen Dokument sieht.

Ein typischer Code-Chunk sieht so aus:

```
```r
#| echo: false
#| warning: false
#| message: false
library(ggplot2)
ggplot(mtcars, aes(x = hp, y = mpg)) + geom_point()
```
```

Die wichtigsten Befehle zur Output-Steuerung:

- `#| echo: false` → Der Code selbst wird im Dokument **ausgeblendet**, aber das Resultat (z. B. der Plot oder die Tabelle) wird **angezeigt**. Das ist essenziell für Berichte, die von Fachfremden oder dem Management gelesen werden!
- `#| eval: false` → Der Code wird **angezeigt**, aber **nicht ausgeführt** (nützlich, wenn Sie nur zeigen wollen, wie ein Befehl aussieht).
- `#| warning: false` → Unterdrückt rote R-Warnmeldungen im finalen Dokument.
- `#| message: false` → Unterdrückt allgemeine Info-Texte von R (z. B. beim Laden von Paketen).
- `#| fig-cap: "Ihre Bildunterschrift"` → Fügt automatisch eine Beschriftung unter Ihre erstellte Grafik ein.

13.4 Best Practices für Abgaben

Halten Sie sich an diese drei eisernen Regeln, um Fehler beim Rendern (Kompilieren) kurz vor der Abgabe-Deadline zu vermeiden:

1. **Der Setup-Chunk gehört nach ganz oben:** Laden Sie alle benötigten Pakete (z. B. `library(dplyr)`) und Ihre Datensätze immer im allerersten Code-Chunk direkt unter dem YAML-Header. Wenn Sie Pakete mitten im Text laden, verlieren Sie schnell den Überblick. Versehen Sie diesen Setup-Chunk immer mit `#| message: false` und `#| warning: false`.

2. **Text und Code strikt trennen:** Schreiben Sie Ihre Schlussfolgerungen und Interpretationen immer im normalen Textbereich (weiß) und niemals als Kommentare (#) versteckt im Code-Bereich (grau).
3. **Regelmäßig rendern:** Drücken Sie nicht erst ganz am Ende Ihres Projekts auf den “Render”-Button. Rendern Sie das Dokument regelmäßig (z. B. nach jedem neuen Kapitel oder komplexen Plot). Wenn ein Fehler auftritt, wissen Sie so sofort, dass er im zuletzt geschriebenen Block stecken muss.

Çetinkaya-Rundel, Mine, und Johanna Hardin. 2024. *Introduction to modern statistics*. 2nd edition. OpenIntro.